



**DEVELOPMENT OF AN IOT BASED SMART WIFI PLUG FOR REMOTE ENERGY  
MONITORING AND CONTROL**

**By**

**IKOKO, MICHAEL OLISE-EKEZIE**

**EEE/20/5669**

**SUBMITTED TO**

**THE DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING**

**THE SCHOOL OF ELECTRICAL SYSTEMS ENGINEERING**

**THE FEDERAL UNIVERSITY OF TECHNOLOGY, AKURE, ONDO STATE, NIGERIA**

**IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE AWARD OF A**

**BACHELOR OF ENGINEERING (B.ENG) DEGREE OF ELECTRICAL AND**

**ELECTRONICS ENGINEERING**

**SUPERVISOR: PROF. A. A PONNLE**

**AUGUST, 2026**

## **CERTIFICATION**

This document serves as confirmation that the technical report presented provides a comprehensive record of the final-year undergraduate project work completed by IKOKO, MICHAEL OLISE-EKEZIE, matriculation number EEE/20/5669, in the department of Electrical and Electronics Engineering, the Federal University of Technology, Akure. The report has been prepared per the regulations governing the production of reports in the department and submitted in partial fulfillment of the requirement for the award of a Bachelor of Engineering (B Eng.) degree in Electrical and Electronics Engineering.

**IKOKO MICHAEL OLISE-EKEZIE**

.....

(Student)

Signature and Date

**Prof. A. A. Ponnle**

.....

(Supervisor)

Signature and Date

**Dr. B. C. Ubochi**

.....

(Head of Department)

Signature and Date

## **DEDICATION**

This project is dedicated to my mother, and to my brothers and sister, for their unwavering love, support, and encouragement throughout my academic journey. .

## **ACKNOWLEDGEMENT**

I am deeply grateful to my mother, Mrs. Philomina Ikoko, and my brothers, Olise, Emeke, and Chukwuka, and my sister, Keonye, for their unwavering love, support, encouragement, and sacrifices throughout my academic journey. Their presence and support have been a constant source of strength and motivation, and I am sincerely grateful for all they have done for me

I am also sincerely grateful to my supervisor, Prof. A. A. Ponnle, for his guidance, patience, support, and constructive feedback throughout the course of this project. His direction and encouragement were invaluable in the successful completion of this work.

## ABSTRACT

Conventional electrical outlets provide no capability for real-time monitoring, remote control, or appliance-level energy visibility, limiting users' ability to identify high consumption devices or manage electricity costs effectively. This project addresses that gap through the design and implementation of an IoT based smart WiFi plug for remote energy monitoring and control, capable of measuring voltage, current, power, frequency, power factor, and accumulated energy in real time, while enabling remote switching of connected appliances via a mobile application. The system was developed using an iterative, prototype driven methodology across four integrated layers: a physical layer combining an ESP32-C3 microcontroller with an optically isolated relay and a PZEM-004T energy monitoring module communicating via UART/Modbus RTU; a network layer utilizing TLS-secured MQTT authenticated through a managed HiveMQ Cloud broker; a FastAPI backend persisting telemetry to a PostgreSQL database and enforcing configurable energy limits with automatic cutoff; and a React Native cross-platform mobile application providing real-time telemetry, historical analytics, and timer-based automation. Empirical evaluation across resistive and electronic household loads demonstrated that the prototype achieved reliable relay switching and stable telemetry reporting, with remote command response latency averaging 1135.5 ms under mobile network conditions and voltage measurements exhibiting a consistent, systematic offset of 1.3% to 1.6% relative to an independent reference multimeter. These results confirm that accurate measurement, secure authenticated communication, reliable switching, and functional remote monitoring can be effectively achieved on low-cost, open hardware, with identified limitations including the absence of inductive load testing and formal network stability profiling carried forward as recommendations for future development.

## **TABLE OF CONTENT**

|                                                      |    |
|------------------------------------------------------|----|
| <b>CERTIFICATION</b>                                 | 1  |
| <b>DEDICATION</b>                                    | 2  |
| <b>ACKNOWLEDGEMENT</b>                               | 3  |
| <b>ABSTRACT</b>                                      | 4  |
| <b>TABLE OF CONTENT</b>                              | 5  |
| <b>LIST OF FIGURES</b>                               | 8  |
| <b>LIST OF PLATES</b>                                | 9  |
| <b>CHAPTER ONE</b>                                   | 1  |
| <b>INTRODUCTION</b>                                  | 1  |
| 1.1 Background Studies                               | 1  |
| 1.2 Problem Statement                                | 2  |
| 1.3 Aim of the Project                               | 3  |
| 1.4 Objectives of the Project                        | 3  |
| 1.5 Scope and Limitations                            | 3  |
| 1.6 Significance of the Study                        | 4  |
| <b>CHAPTER TWO</b>                                   | 6  |
| <b>LITERATURE REVIEW</b>                             | 6  |
| 2.1 IoT and Smart Home Foundational Concepts         | 6  |
| 2.2 Related Smart-Plug Systems                       | 7  |
| 2.2.1 Sensing Hardware                               | 8  |
| 2.2.2 Communication Approach                         | 8  |
| 2.2.3 Safety and Cutoff Mechanisms                   | 9  |
| 2.2.4 Intelligence and Scheduling                    | 9  |
| 2.2.5 Real-World and Longitudinal Evaluation         | 10 |
| 2.2.6 Common Limitations Across Reviewed Systems     | 10 |
| 2.3 Broader Smart-Home and Energy-Monitoring Systems | 12 |
| 2.4 ESP32/Microcontroller Platforms                  | 14 |
| 2.5 Energy Sensing                                   | 17 |
| 2.6 Communication Protocols                          | 18 |
| 2.6.1 Protocol-Level Comparisons                     | 18 |

|                                                          |    |
|----------------------------------------------------------|----|
| 2.6.2 MQTT Transport and Configuration Tradeoffs         | 19 |
| 2.6.3 MQTT Power Efficiency                              | 20 |
| 2.6.4 Broker Architecture and Scalability                | 20 |
| 2.7 Backend/Remote-Control Architecture                  | 21 |
| 2.8 IoT Security                                         | 23 |
| 2.8.1 Scale and Severity of the Problem                  | 23 |
| 2.8.2 Network and Communication Vulnerabilities          | 23 |
| 2.8.3 Firmware and Hardware-Level Vulnerabilities        | 24 |
| 2.8.4 Recommended Mitigations                            | 24 |
| 2.8.5 Relevance to This Project                          | 25 |
| 2.9 Research Gap and Summary                             | 25 |
| CHAPTER THREE                                            | 27 |
| METHODOLOGY                                              | 27 |
| 3.1 Development Methodology                              | 27 |
| 3.2 System Overview                                      | 27 |
| 3.3 Hardware Design and Implementation                   | 28 |
| 3.3.1 Component Selection                                | 28 |
| 3.3.2 Power Supply Circuit                               | 31 |
| 3.4 Measurement and Control Subsystem                    | 32 |
| 3.4.1 Voltage Sensing                                    | 32 |
| 3.4.2 Current Sensing (Toroidal CT Transformer)          | 33 |
| 3.4.3 Power, Power Factor, and Energy Calculation        | 34 |
| 3.4.4 PZEM-004T Communication Protocol                   | 36 |
| 3.4.5 Relay Control                                      | 36 |
| 3.5 Firmware Architecture                                | 37 |
| 3.5.1 Development Environment                            | 37 |
| 3.5.2 Device Operating Modes                             | 37 |
| 3.6 Communication Protocol                               | 38 |
| 3.6.1 Protocol Selection                                 | 38 |
| 3.6.2 MQTT Topic Structure                               | 38 |
| 3.7 Cloud Infrastructure, Backend and Mobile Application | 41 |

|                                                      |    |
|------------------------------------------------------|----|
| 3.7.1 MQTT Broker                                    | 41 |
| 3.7.2 Backend Server                                 | 41 |
| 3.7.3 Database Design                                | 41 |
| 3.7.4 Mobile Application Screens                     | 43 |
| 3.8 Testing and Evaluation Methodology               | 44 |
| 3.8.1 Measurement Accuracy                           | 44 |
| 3.8.2 Remote Control Response Time                   | 45 |
| 3.8.3 Reliability Under Varying Load Conditions      | 45 |
| CHAPTER FOUR                                         | 46 |
| RESULTS AND DISCUSSION                               | 46 |
| 4.1 Introduction                                     | 46 |
| 4.2 Hardware Implementation                          | 46 |
| 4.3 Firmware and Software Implementation             | 48 |
| 4.4 Test Environment and Equipment                   | 52 |
| 4.5 Measurement and Performance Evaluation           | 52 |
| 4.5.1 Voltage Measurement Accuracy                   | 52 |
| 4.5.2 Load Measurement                               | 54 |
| 4.6 Remote Control Response Time                     | 56 |
| 4.7 System Reliability Under Varying Load Conditions | 57 |
| 4.8 Discussion of Results Against Project Objectives | 57 |
| 4.9 Limitations of Testing                           | 59 |
| CHAPTER FIVE                                         | 61 |
| CONCLUSION AND RECOMMENDATIONS                       | 61 |
| 5.1 Conclusion                                       | 61 |
| 5.2 Recommendations for Future Work                  | 62 |
| REFERENCES                                           | 63 |

## LIST OF FIGURES

| Figure | Description                                                          | Page Number |
|--------|----------------------------------------------------------------------|-------------|
| 2.1    | Functional block diagram of PZEM-004T-100A energy measurement module | 8           |
| 3.1    | Overall system architecture                                          | 28          |
| 3.2    | ESP32-C3 Mini                                                        | 29          |
| 3.3    | PZEM-004T Module                                                     | 29          |
| 3.4    | HLK-PM05                                                             | 29          |
| 3.5    | 1-Channel Relay Module                                               | 29          |
| 3.6    | Current Transformer                                                  | 30          |
| 3.7    | Fuse                                                                 | 30          |
| 3.8    | Varistor                                                             | 30          |
| 3.9    | Class X2 Capacitor                                                   | 30          |
| 3.10   | Common Mode Choke                                                    | 30          |
| 3.11   | Output Capacitor                                                     | 31          |
| 3.12   | Circuit Schematic                                                    | 32          |
| 3.13   | Operating Modes                                                      | 38          |
| 4.1    | API Swagger UI Documentation Screenshot                              | 48          |
| 4.2    | Login screen                                                         | 49          |
| 4.3    | Add device screen                                                    | 49          |
| 4.4    | Home screen                                                          | 49          |
| 4.5    | Device overview screen                                               | 49          |
| 4.6    | Device automation screen                                             | 50          |
| 4.7    | Device settings screen                                               | 50          |
| 4.8    | Fleet analytics screen                                               | 50          |
| 4.9    | Alerts screen                                                        | 50          |
| 4.10   | Profile screen                                                       | 51          |
| 4.11   | Energy limit triggered                                               | 51          |
| 4.12   | Timer triggered                                                      | 51          |
| 4.13   | Device weekly usage screen                                           | 51          |

## **LIST OF PLATES**

| <b>Plate</b> | <b>Description</b>                                                | <b>Page Number</b> |
|--------------|-------------------------------------------------------------------|--------------------|
| 2.1          | ESP32-C3 SuperMini Development Board and Pinout Configuration     | 15                 |
| 4.1          | Populated perfboard before enclosure, showing component placement | 46                 |
| 4.2          | Fully assembled unit inside the single-gang back box              | 47                 |
| 4.3          | Assembled unit connected and powered, ready for testing           | 47                 |

## LIST OF TABLES

| Table | Description                                                                                                                                                                              | Page Number |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| 2.1   | Comparative summary of the smart plug systems reviewed in this section, columns for Author and Year, Sensor Type, Communication Protocol, Reported Accuracy, and Cutoff/Safety Mechanism | 11          |
| 3.1   | Table of components used, specification, justification, and Image.                                                                                                                       | 29          |
| 3.2   | PZEM-004T Modbus register map for measurement results (PZEM-004T V3.0 User Manual, n.d.)                                                                                                 | 36          |
| 3.3   | MQTT Topic Structure                                                                                                                                                                     | 39          |
| 3.4   | Payload of MQTT Topics                                                                                                                                                                   | 40          |
| 3.5   | Database tables, columns and purpose                                                                                                                                                     | 41          |
| 3.6   | Mobile Application Screens                                                                                                                                                               | 43          |
| 4.1   | Test equipments used and environment                                                                                                                                                     | 52          |
| 4.2   | Soldering iron reference versus system voltage                                                                                                                                           | 53          |
| 4.3   | Pressing iron reference versus system voltage                                                                                                                                            | 53          |
| 4.4   | Phone charger reference versus system voltage                                                                                                                                            | 54          |
| 4.5   | Voltage error statistics across all three loads                                                                                                                                          | 54          |
| 4.6   | Soldering iron(rated 60W, 220V) system measurement                                                                                                                                       | 55          |
| 4.7   | Pressing iron(rated 1200W, 220 to 240V) system measurement                                                                                                                               | 55          |
| 4.8   | Phone charger(rated 12W, 180 to 240V) system measurements                                                                                                                                | 56          |
| 4.9   | Command response time                                                                                                                                                                    | 56          |
| 4.10  | System reliability under various loads                                                                                                                                                   | 57          |
| 4.11  | Summary of project results against objectives                                                                                                                                            | 58          |

## CHAPTER ONE

### INTRODUCTION

#### 1.1 Background Studies

The Internet of Things (IoT) has extended automation and remote management into the domestic environment, connecting everyday devices, sensors, and mobile users over the internet to enable monitoring and control from any location at any time (Hassan and Eassa, 2022). Within this ecosystem, smart plugs have emerged as a particularly practical entry point: rather than requiring users to replace existing appliances, they act as a retrofitting layer that adds connectivity and control to devices that were never designed to be "smart" (Gomes *et al.*, 2019). This retrofit model, often described as a plug-and-play approach, is frequently cited as a key reason smart plugs have helped popularize the broader smart-home concept, since it lowers the cost and complexity of adoption compared to replacing an entire appliance fleet (Suryadevara and Biswal, 2019).

Market indicators reflect this growth. The global smart plug market's annual growth rate was estimated at around 38% between 2017 and 2021 (Suryadevara and Biswal, 2019), while smart-home device deployment as a whole was projected to reach 216.9 million homes worldwide by 2022 (Gomes *et al.*, 2019). This growth is closely tied to energy-management motivations. Gomes *et al.* (2019) reported that in a residential case study, air conditioning alone accounted for 72% of total household energy consumption during the monitored period, and that basic scene-based smart plug automation reduced consumption by up to 25.5%, rising to 35.6% when context-aware Environmental Awareness Plug (EnAPlug) devices were used instead. This difference translates to roughly 31.02 EUR in monthly savings under 2017 EU electricity pricing. Comparable results appear elsewhere in the literature: occupancy-based automation has been shown to cut office energy consumption by 10 to 15% (Suryadevara and Biswal, 2019), and a smart-office deployment combining Bluetooth Low Energy beacons with smart plugs achieved average savings of 31.3% for PCs and 15.3% for lighting (Suryadevara and Biswal, 2019).

Beyond direct energy savings, smart home automation is also framed in the literature as reducing the time, cost, and daily effort required from occupants, with context-aware systems able to adapt services, such as adjusting climate control or shutting down irrigation in response to rain, without manual intervention (Hassan and Eassa, 2022). Taken together, these findings position the smart

plug not as a novelty device but as one of the more accessible, evidence-backed mechanisms for reducing residential energy waste at scale.

However, this growth and demonstrated potential also expose a gap between what the literature shows is possible and what is typically available in commodity smart plug products, a tension this project's problem statement addresses directly.

## **1.2 Problem Statement**

Despite the demonstrated energy-saving potential of smart plug systems, empirical evaluation of commercially available devices reveals significant and largely unaddressed reliability gaps. Santos *et al.* (2023) tested five popular smart plug brands under variable electrical loads and found substantial measurement inaccuracies, with maximum energy-measurement errors ranging from 54% to 101% for four of the five brands tested. Beyond measurement accuracy, the same study identified significant delays between a load-state change and the device reporting it, averaging 3 to 6 seconds and reaching as high as 20 seconds in some cases, a lag serious enough to undermine both real-time monitoring and any automated control logic that depends on timely state information. The study further noted that several models' internal power draw increased substantially under heavy load, in some cases exceeding manufacturer specifications several times over, meaning the monitoring device itself becomes a non-trivial source of the very consumption it is meant to track.

These reliability concerns are compounded by security weaknesses documented broadly across low-end IoT device operating systems. Al-Boghdady *et al.* (2021) found persistent, exploitable vulnerabilities, including memory corruption and buffer overflow defects, across widely used open-source IoT operating systems, correlating error density with codebase size and developer experience rather than any single fixable defect. The consequences of such vulnerabilities are not merely theoretical: insecure IoT devices have been documented as the basis for large-scale botnet attacks, and a 2014 industry study found 70% of IoT devices vulnerable to attack, averaging 25 vulnerabilities per device.

Taken together, these findings point to a consistent gap between the promise of smart plug technology and the reliability of what commodity and prototype systems actually deliver: unstable measurement under real-world loads, response delays that compromise timely control, and security

postures that leave devices open to compromise. A smart plug system intended for practical, sustained use must therefore be evaluated not only on whether it can monitor and control energy consumption, but on whether it does so accurately, promptly, and securely, the three dimensions this project addresses directly in its design and implementation.

### **1.3 Aim of the Project**

This project aims to design and develop an IoT-based smart WiFi plug system that enables users to remotely monitor electrical parameters (voltage, current, power, frequency, power factor, and energy consumption) and control household appliance in real time.

### **1.4 Objectives of the Project**

In order to achieve this aim, the project objectives are to:

- a. Design and implement ESP32-C3-based firmware capable of switching appliance power state via relay control, with consistent state persistence across both remote (MQTT) and local (physical button) control paths.
- b. Establish secure, authenticated device-to-backend communication using MQTT over TLS, in place of the unauthenticated public brokers common in prototype-stage IoT systems.
- c. Develop a backend service for device registration, telemetry storage, timer-based automation, and configurable energy-limit enforcement with automatic cutoff.
- d. Develop a cross-platform mobile application providing real-time device monitoring, historical energy consumption analytics, and remote control of connected devices.
- e. Test and evaluate the developed system's functionality, reliability, and security under real hardware operation.

### **1.5 Scope and Limitations**

This project covers the end-to-end design and implementation of a functional smart WiFi plug prototype, comprising three integrated components: an ESP32-C3-based firmware layer with relay-based switching and real-time energy sensing via a PZEM current/voltage sensing module; a FastAPI backend handling device registration, telemetry storage, timer automation, and energy-limit enforcement; and a cross-platform mobile application for remote monitoring and control. The system is designed for single-appliance, per-plug monitoring within a residential WiFi network, with device-to-cloud communication over MQTT secured via TLS.

The scope is bounded to a single-user ownership model per device; multi-user or shared-household access control is not addressed. Communication is limited to WiFi-based connectivity; alternative low-power protocols such as Zigbee or LoRaWAN, while reviewed in the literature, fall outside the implemented system. Automation is limited to timer-based scheduling and threshold-based cutoff; predictive or machine-learning-based load forecasting is not implemented.

Several limitations are acknowledged as accepted trade-offs given project timeline constraints rather than as unaddressed design flaws. The mobile application currently embeds MQTT broker credentials client-side, since the app connects directly to the broker rather than through a backend-mediated relay; while this is a meaningful improvement over an unauthenticated public broker, a backend-proxied MQTT architecture would be the more robust long-term approach. Over-the-air (OTA) firmware updates are not implemented, meaning firmware updates require physical access to the device. Additionally, the system has been developed and tested on a single hardware unit; multi-device scalability under concurrent load has not been evaluated at production scale.

## **1.6 Significance of the Study**

This project contributes to the growing body of practical work on affordable, self-hosted IoT energy monitoring by demonstrating a complete, end-to-end smart plug system built on low-cost, widely available components (ESP32-C3, PZEM sensing, open-source backend and mobile frameworks), rather than relying on proprietary commercial hardware or vendor-locked cloud platforms. This positions the system as a reproducible reference implementation that other students, hobbyists, or small-scale developers can build on or adapt.

Beyond its technical contribution, the project directly addresses two categories of shortcoming repeatedly identified in the literature on existing smart plug and IoT systems: unreliable measurement and delayed state reporting under real-world conditions (Santos *et al.*, 2023), and insecure device-to-broker communication stemming from unauthenticated or poorly configured connections (Al-Boghdady *et al.*, 2021). By implementing TLS-secured, authenticated MQTT communication, explicit connection-loss detection, and consistent state persistence across both remote and local control paths, the system offers a concrete demonstration that these commonly cited gaps can be closed within a modest, single-developer project timeline; evidence relevant to students and early-stage developers who may otherwise treat such issues as acceptable prototype-stage shortcuts.

For end users, the system offers practical value in the form of granular, real-time energy visibility and automated cutoff control, supporting more informed household energy consumption decisions and potentially contributing to the kind of measurable savings reported elsewhere in the literature for automated and context-aware energy management. For the researcher, the project provides direct, hands-on experience spanning embedded firmware development, backend API and database design, secure network communication, and mobile application development, an integration of skills directly relevant to IoT systems engineering practice.

## CHAPTER TWO

### LITERATURE REVIEW

#### 2.1 IoT and Smart Home Foundational Concepts

The Internet of Things (IoT) is broadly defined as a computing and communications paradigm in which everyday objects are augmented with computational and wireless communication capabilities, typically through resource-constrained devices such as sensors and actuators, enabling their connection to the internet (Gomez *et al.*, 2019). In the context of the home, this translates into the ability to remotely manage mobile users, devices, and sensors over an internet connection, with the broader goal of allowing objects to connect with any user, at any time and place (Hassan and Eassa, 2022). IoT is therefore not itself a single technology but the underlying paradigm that makes smart environments possible (including smart homes, smart health, smart cities, and smart factories), by providing the means to acquire information from the physical world, process it, and act on it (Gomez *et al.*, 2019).

Gomez *et al.* (2019) note that different smart-environment domains adopt IoT technologies at markedly different paces: personal-oriented domains such as smart homes evaluate and integrate new technology relatively quickly, while more complex domains such as smart factories and smart cities are slowed by legacy infrastructure. This distinction carries architectural consequences. Smart homes typically rely on dedicated and smartphone-based sensors over mesh or star network topologies, and, because their latency requirements are comparatively relaxed, predominantly use cloud computing for data processing, in contrast to smart factories, which require fog (edge) computing to meet millisecond-level, deterministic control demands (Gomez *et al.*, 2019). This positions the residential smart plug domain as one where a cloud-connected, internet-dependent architecture is both technically appropriate and consistent with common practice, rather than a simplification made purely for convenience.

Within this domain, smart home systems are generally framed as automated solutions intended to save time, reduce owner effort, and lower operating costs, often incorporating context-aware computing: the system's ability to gather environmental context and respond with flexible, adaptive services, such as automatically adjusting climate control or halting irrigation in response to rain (Hassan and Eassa, 2022). Smart plugs represent a specific, narrower instantiation of this broader smart-home paradigm. Rather than requiring dedicated smart appliances, they function as

retrofitting devices that convert existing "dumb" appliances into IoT-connected nodes, forming what is described in the literature as a Hybrid Appliance Load Monitoring System (Suryadevara and Biswal, 2019). Their value lies substantially in this plug-and-play accessibility: because they avoid the need to replace existing electrical equipment, they are considered a significant enabler in popularizing smart-home adoption more broadly (Suryadevara and Biswal, 2019).

A recurring technical challenge noted across this literature is the risk of standards lock-in. Gomez *et al.* (2019) observe that although the coexistence of multiple communication standards and protocols reflects a highly active and evolving market, the lack of straightforward interoperability between them can lock users into a given standard that may become obsolete, underscoring the importance of deliberate, justified protocol selection at the design stage rather than default adoption of whatever standard is most common; this is a consideration directly relevant to the communication protocol choices examined later in this review (see Section 2.6).

Taken together, these foundational concepts establish the smart plug not as an isolated gadget but as a specific, well-motivated application of the broader IoT and smart-home paradigm: a cloud-connected, retrofitting device operating within a domain that favors rapid technology adoption, relaxed latency tolerance, and internet-dependent architecture. The sections that follow narrow this foundation into the specific systems, hardware platforms, and protocols relevant to this project's design.

## **2.2 Related Smart-Plug Systems**

A substantial body of applied research has produced working smart plug prototypes that combine current/voltage sensing, wireless connectivity, and cloud- or app-based control. Several systems follow a broadly similar four-layer pattern: a perception layer (the sensing/switching hardware), a network layer (Wi-Fi or an alternative radio), a data layer (a cloud database or broker), and an application layer (a mobile or web dashboard), built around commercial Wi-Fi smart plugs paired with MQTT-based cloud infrastructure and Android applications for consumption tracking and appliance identification (Albraheem *et al.*, 2023; Dutta *et al.*, 2023). Others replace the commercial plug with a self-built prototype centered on low-cost microcontrollers such as the ESP8266, ESP32, or Arduino-class boards.

### 2.2.1 Sensing Hardware

Accurate current and voltage sensing is foundational to any smart plug system, and the reviewed literature converges on two dominant approaches. The PZEM-004T current/voltage sensor recurs across multiple independent designs, consistently reported as accurate to well under 1% error (Hasanah *et al.*, 2025; Sánchez-Sutil and Cano-Ortega, 2022; Gde *et al.*, 2025), while several other systems instead use ACS712 Hall-effect current sensors paired with a relay for switching (Blanco-Novoa *et al.*, 2017; Bajracharya *et al.*, 2025; Elorbany *et al.*, 2021). Reported accuracy across both approaches is generally high; for example, Hasanah *et al.* (2025) report an overall system accuracy of 99.95%, and Bajracharya *et al.* (2025) report current accuracy of  $\pm 0.08$  A, suggesting that basic measurement accuracy is a largely solved problem at the component level, in contrast to the load-dependent measurement instability reported for commercial devices under variable loads (Santos *et al.*, 2023). The functional architecture of the PZEM-004T module is illustrated in Figure 2.1.

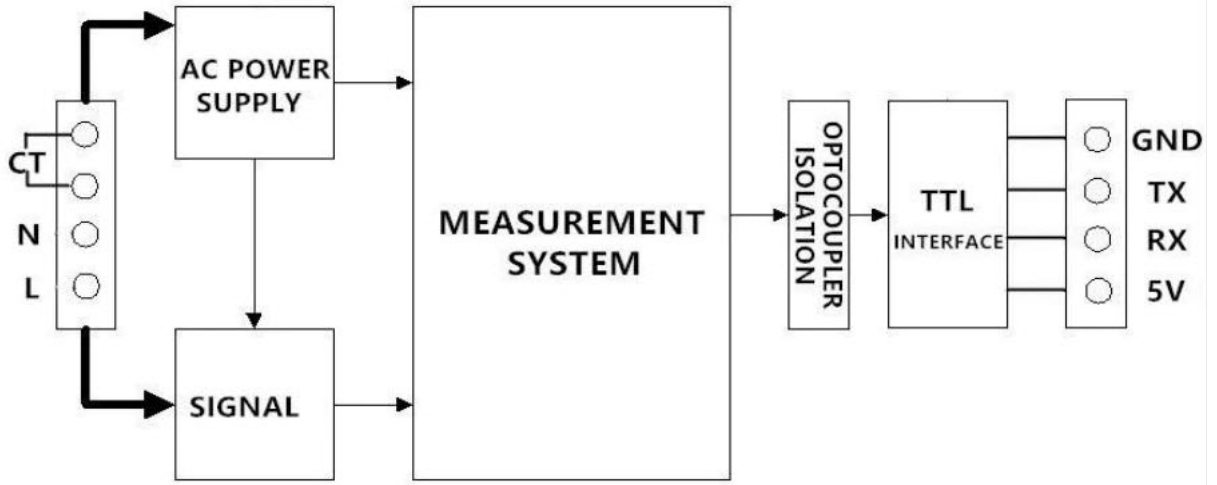


Figure 2.1: Functional block diagram of PZEM-004T-100A energy measurement module

### 2.2.2 Communication Approach

While Wi-Fi dominates as the connectivity choice among reviewed systems, several designs adopt alternative protocols to address specific range, resolution, or processing constraints. Sánchez-Sutil and Cano-Ortega (2022) adopt LoRaWAN, reporting coverage up to 5 km without repeaters and a temporal resolution of 0.5 seconds, a notable improvement over the 1 to 15 minute intervals typical of commercial devices, at the cost of requiring a separate LoRaWAN gateway with its own internet connection. Dhaou (2023) instead builds a Zigbee-based fog-computing architecture,

offloading appliance identification to a Raspberry Pi smart-meter node using TinyML classifiers, achieving smart-plug accuracy above 99% and inference latency in the microsecond range. These alternative-protocol systems consistently report performance advantages over plain Wi-Fi in their specific target metric (range, resolution, or edge-processing latency), but each introduces additional architectural complexity (a gateway, a coordinator node, or an edge-inference pipeline) not required by simpler Wi-Fi-direct-to-cloud designs.

### **2.2.3 Safety and Cutoff Mechanisms**

Several reviewed systems implement automatic protective cutoff to prevent overcurrent or overheating damage, a feature directly relevant to this project's own energy-limit enforcement. Hasanah *et al.* (2025) trip a relay within 15 seconds of current exceeding 3.1 A; Bajracharya *et al.* (2025) cut power at 10 A or 70 degrees Celsius internal temperature and additionally auto-disconnect idle outlets after 30 seconds of zero current draw; Elorbany *et al.* (2021) implement software-configurable overcurrent thresholds set through a mobile app. None of these designs, however, report how their systems behave when the cutoff decision must be made locally during a network outage; this is a gap this project's own local/remote state-consistency design was built to address.

### **2.2.4 Intelligence and Scheduling**

A smaller cluster of systems extends beyond basic monitoring and cutoff into automated decision-making. Blanco-Novoa *et al.* (2017) implement price-aware scheduling that automatically shifts appliance operation to low-tariff periods by polling a national electricity-pricing API, reporting cost savings of up to 24.33% on high-load devices, though at a measured cryptographic cost: HTTPS transactions required roughly three times the processing time of plain HTTP on their constrained hardware. At the more sophisticated end, Gomes *et al.* (2018, 2019) present EnAPlug, a multi-agent, context-aware smart plug system using local deep-learning models to forecast occupancy and pre-condition appliances such as air conditioners and heaters, reporting appliance-specific forecast accuracies between 87% and 92% and combined household-presence certainty of 99.94% when multiple plugs share predictions. Their comparative case study found EnAPlug's autonomous optimization saved 35.6% of energy against a manual-control baseline, compared to 25.5% for conventional smart plugs using manually configured schedules, a meaningful gap attributable specifically to the added context-awareness layer, though the authors note this comes

at a prototype cost of approximately 130 EUR, driven largely by the power supply and external energy-analyzer hardware.

### **2.2.5 Real-World and Longitudinal Evaluation**

Most reviewed systems report only short-term lab or bench validation, which limits confidence in how well their reported gains hold up under sustained, real-world use. A notable exception is Oh (2020), which evaluated a combined smart-plug-and-education program across 125 households over 15 months, finding a statistically significant 3.53% average reduction in monthly electricity use, modest in percentage terms, translating to only around \$1.20 in monthly savings per household, which the author notes is insufficient to justify voluntary device purchase without external subsidy. This finding is a useful counterweight to the higher savings percentages reported in shorter, more controlled studies elsewhere in this section, and suggests that a system's technical capability does not automatically translate into large real-world savings without a sustained behavioral or educational component.

### **2.2.6 Common Limitations Across Reviewed Systems**

Several gaps recur regardless of a system's specific architecture. Prototype-stage systems frequently lack manual or voice control entirely (Albraheem *et al.*, 2023) or remain confined to bench-level validation without a path to commercialization (Khadar *et al.*, 2024). Cloud-dependent designs are consistently reported as losing all local control capability during internet outages (Bajracharya *et al.*, 2025), and several authors flag security and privacy concerns, including device spoofing and data exposure, as unresolved (Dutta *et al.*, 2023). Even the most capable system reviewed, EnAPlug, acknowledges a fragmented, non-standardized smart-plug market as an ongoing barrier to third-party interoperability (Gomes *et al.*, 2019).

Across this body of work, a consistent pattern emerges: systems that are simple and low-cost tend to under-specify security, offline behavior, and long-term reliability, while systems that address these gaps do so by adding substantial architectural or hardware complexity (fog nodes, external gateways, or machine-learning pipelines) that raises cost and implementation difficulty well beyond a single-developer, single-semester scope. This project's design occupies a comparatively under explored middle ground: a Wi-Fi-direct-to-cloud architecture using authenticated, TLS-secured MQTT rather than the unauthenticated or unspecified security postures common across the systems reviewed here, combined with explicit local-fallback state handling during connection

loss, addressing a gap none of the reviewed systems report resolving directly. A comparative summary of the reviewed smart plug systems is presented in Table 2.1.

Table 2.1: Comparative summary of the smart plug systems reviewed in this section, columns for Author and Year, Sensor Type, Communication Protocol, Reported Accuracy, and Cutoff/Safety Mechanism

| Author and Year                      | Platform                       | Sensor Type                        | Communication Protocol           | Reported Accuracy and Performance                                                    |
|--------------------------------------|--------------------------------|------------------------------------|----------------------------------|--------------------------------------------------------------------------------------|
| Santos <i>et al.</i> (2023)          | Proprietary commercial SoCs    | Internal shunt / ADC sampling      | Wi-Fi (Vendor Cloud / Local API) | 54% to 101% max error under variable loads (Belkin <1.7%)                            |
| Hasanah <i>et al.</i> (2025)         | NodeMCU (ESP8266)              | PZEM-004T                          | Wi-Fi (Blynk Cloud)              | 99.95% overall accuracy (<0.05% measurement error)                                   |
| Sánchez-Sutil and Cano-Ortega (2022) | Arduino Nano                   | PZEM-004T                          | LoRaWAN (Dragino LoRa Bee / TTN) | <1% measurement error (0.5 s sampling interval)                                      |
| Gde <i>et al.</i> (2025)             | NodeMCU (ESP8266)              | PZEM-004T v3.0                     | Wi-Fi (Blynk Cloud)              | High / consistent readings across LCD and cloud dashboard                            |
| Dhaou (2023)                         | Arduino Uno and Raspberry Pi 3 | ACS712 current and voltage divider | Zigbee (Mesh coordinator/router) | >99% plug accuracy (TinyML edge classifier on Pi)                                    |
| Bajracharya <i>et al.</i> (2025)     | ESP32                          | ACS712 + DHT22 temperature sensor  | Wi-Fi (Arduino IoT Cloud)        | Current: $\pm 0.08$ A; temperature: $\pm 1^\circ\text{C}$ (3rd-order polynomial fit) |
| Elorbany <i>et al.</i> (2021)        | WClip / Wemos D1               | ACS712 + ADS1115 (16-bit)          | Wi-Fi (Firebase Cloud)           | Calibrated RMS current sampling                                                      |

| Author and Year                   | Platform                             | Sensor Type                            | Communication Protocol              | Reported Accuracy and Performance             |
|-----------------------------------|--------------------------------------|----------------------------------------|-------------------------------------|-----------------------------------------------|
|                                   | Mini (ESP8266)                       | bit ADC)                               |                                     |                                               |
| Blanco-Novoa <i>et al.</i> (2017) | Wemos D1 Mini NodeMCU                | Allegro ACS712-20AT                    | Wi-Fi (HTTP / HTTPS REST Proxy)     | Mean-squared sampled current                  |
| Gomes <i>et al.</i> (2018, 2019)  | Single-Board Computer (SBC)          | Modbus energy analyzer + multi-sensors | Wi-Fi / TCP/IP (JADE MAS Framework) | 87%–92% appliance forecast accuracy           |
| Albraheem <i>et al.</i> (2023)    | Mokosmart Plug                       | Internal power metering IC             | Wi-Fi / BLE (MQTT to AWS EC2)       | Accurate vs. manufacturer appliance ratings   |
| Choi <i>et al.</i> (2015)         | Embedded smart outlet                | Internal current sensor                | Bluetooth Low Energy (BLE)          | 31.3% PC / 15.3% lighting energy savings      |
| Oh (2020)                         | Commercial smart plugs               | Internal current/voltage sensing       | Wi-Fi / Cellular                    | 3.53%–5% empirical household energy reduction |
| Khadar <i>et al.</i> (2024)       | Arduino Nano / ATmega328 P / ESP8266 | NTC Thermistor / Time switch           | Bluetooth (HC-05) and Wi-Fi (Blynk) | Bench - validated functional switching        |

### 2.3 Broader Smart-Home and Energy-Monitoring Systems

Beyond dedicated smart plugs, a related body of work applies similar sensing and control architectures to broader home automation and standalone energy-metering systems, offering useful points of comparison even where the target hardware differs from a single-appliance plug.

Several of these systems replicate the PZEM-based metering approach seen in Section 2.2, applied instead to whole-home or standalone meter contexts. Nebrida *et al.* (2023) paired a PZEM-016

module with dual RS-485 converters feeding both a mobile dashboard and a physical LCD, validating the design over a 48-hour household test in which the cumulative energy reading matched the utility billing meter almost exactly. Sk (2022) achieved a comparable result using a PZEM-004T sensor and a Raspberry Pi web server, reporting an average measurement deviation of under 2% across resistive and inductive test loads, and explicitly noted that many existing IoT energy monitors in the literature suffer from inaccurate analog sensor loops or remain untested theoretical designs; this is a reminder that hardware validation, not just circuit design, is what separates a working system from a paper concept. Mhetre *et al.* (2026) similarly reported "industrial-grade" accuracy from a PZEM-004T V3.0 module paired with an ESP32, though without disclosing a specific error figure. Taken together with the PZEM accuracy findings already noted in Section 2.2.1, this reinforces PZEM-family sensors as a well-validated, low-risk choice for the sensing layer of a project at this scale.

Broader home-automation systems in this cluster extend control beyond a single appliance category and frequently incorporate secondary safety sensing. K (2024) combined current sensing with PIR-based occupancy detection and GSM-based SMS alerts to automate appliance switching based on both electrical thresholds and human presence, while also cataloguing a set of literature-reported weaknesses in current IoT energy frameworks, including a lack of situational awareness in sensors, poor interoperability between competing IoT standards, and susceptibility to cybersecurity threats such as denial-of-service and man-in-the-middle attacks. Bhosale *et al.* (2022) took a safety-first framing, pairing appliance switching with flame-sensor-based fire detection and buzzer alerts, and explicitly scoped a substantial future-work list, including cross-platform app support, removing hardcoded device limits, and adding daylight- and motion-based automation, that reflects how much functionality even a "complete" prototype in this space typically still lacks at initial release. Unisa (2022) reported that command latency in their MicroPython-based NodeMCU system was "well within allowable limits," but, notably, did not report a specific delay figure, which limits how directly this finding can be compared against timing figures reported elsewhere in this review.

A recurring theme across this cluster is the tension between technical capability and practical adoption. Unisa (2022) specifically observed that elderly and disabled users, a group with clear potential to benefit from home automation, often resist adoption due to system complexity and

market pricing, a concern that echoes the affordability and financial-payback limitations already noted for smart plug systems in Section 2.2.5. This suggests that usability and cost barriers to adoption are not specific to any one sub-category of smart home device, but a broader characteristic of the domain that this project's own affordability-conscious hardware choices (ESP32-C3, PZEM sensing) are positioned to help mitigate.

## 2.4 ESP32/Microcontroller Platforms

Microcontroller platform choice underlies every system reviewed in this chapter, and the ESP32 family recurs as the dominant choice across the literature, not by default, but for reasons the reviewed sources make explicit. The ESP32 is consistently described as a low-cost, feature-rich system-on-chip combining a dual-core Xtensa LX6 processor (configurable up to 240 MHz), up to 520 KB of internal SRAM, and integrated Wi-Fi (802.11 b/g/n) and Bluetooth/BLE connectivity in a single package (Bhaganagare *et al.*, 2025a; Hercog *et al.*, 2023). Its dual-core architecture is specifically highlighted as functionally important rather than incidental: Bhaganagare *et al.* (2025b) note that dedicating one core to network stack management (Wi-Fi/MQTT) while the other handles application logic prevents the processing lag and command timeouts observed on single-core platforms. This justifies the ESP32's selection over several named alternatives across the reviewed papers: Arduino Uno-based setups, criticized for lacking built-in wireless connectivity and running at a fraction of the clock speed (Bhaganagare *et al.*, 2025a; Bhaganagare *et al.*, 2025b); the ESP8266/NodeMCU, faulted for its single-core architecture and network delays under concurrent load (Bhaganagare *et al.*, 2025b); the Raspberry Pi, criticized for higher cost, higher power draw, and greater software complexity (Bhaganagare *et al.*, 2025b); and Bluetooth-only or FPGA-based systems, limited by short range and poor multi-device scalability (Aarti *et al.*, 2026). Some designs favor specific ESP32 variants for added capability; Hercog *et al.* (2023), for instance, selected the TTGO T8 module for its onboard microSD slot, 8 MB PSRAM, and integrated battery-charging circuit, extending the base chip for data-logging use cases. The ESP32-C3 SuperMini development board used in this project and its pin configuration are shown in Figure 2.2.

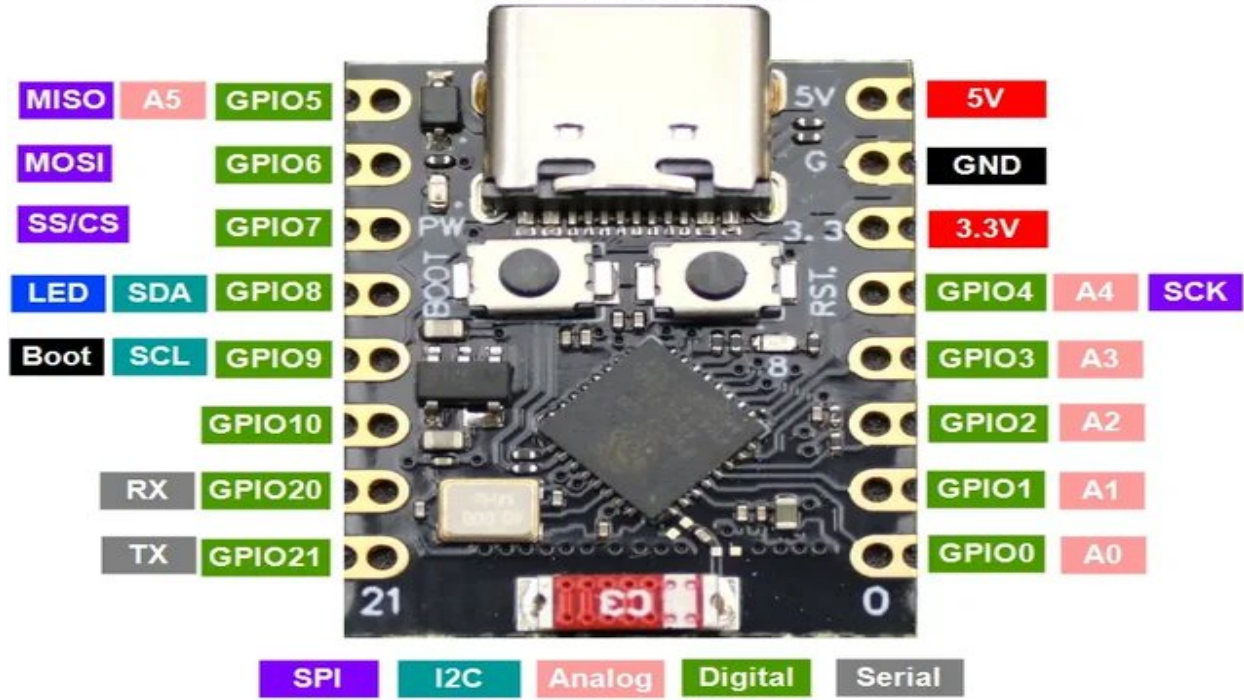


Plate 2.1: ESP32-C3 SuperMini Development Board and Pinout Configuration

Across the reviewed systems, the ESP32 most commonly serves as the central controller and communication bridge, coordinating sensor input and actuator output while maintaining the network connection to a cloud service. Sensor integration is broad and recurring: DHT11/DHT22 for temperature and humidity, PIR for motion, MQ2/MQ135 for gas and air quality, and LDR for light level appear repeatedly across systems (Aarti *et al.*, 2026; Ahmed *et al.*, 2024; Anusha, 2024), typically paired with relay modules for high-voltage appliance switching. Communication is predominantly Wi-Fi-based, most often carried over MQTT to a cloud broker or platform such as Adafruit IO, Firebase, or Blynk (Bhaganagare *et al.*, 2025b; Aarti *et al.*, 2026; Anusha, 2024). Notably, one system extends the ESP32's role beyond conventional home automation: Villegas *et al.* (2025) use it as a bridge between a voice assistant and an industrial Siemens LOGO! PLC, converting voice-triggered Wi-Fi commands into GPIO pulses the PLC can read, while adding optocouplers to isolate the ESP32's 3.3V logic from the PLC's 12 to 24V inputs and wiring manual pushbuttons in parallel for full operational redundancy if the ESP32 or Wi-Fi connection fails. This hybrid approach illustrates a broader principle relevant to this project's own design: treating

the ESP32 as a coordination layer rather than the sole point of control, with a hardware fallback path preserved for when the smart layer is unavailable.

Reported performance figures vary by what each study chose to measure, but several benchmarks are directly comparable to this project's own concerns. Bhaganagare *et al.* (2025b) reported an average voice-command response time of 0.85 seconds and a 98.7% command success rate over a 48-hour test, with most failures attributed to Wi-Fi drops rather than ESP32 processing issues, alongside 99.2% system uptime aided by automatic recovery firmware. Villegas *et al.* (2025) reported a 100% success rate for manual pushbutton control versus over 95% for voice-triggered control through the ESP32, with the gap attributed to network latency and voice-recognition variability rather than the microcontroller itself. This is a pattern consistent with the broader finding that connectivity, not the ESP32 hardware, is typically the weaker link. On power and cost, Hercog *et al.* (2023) measured average current draw of approximately 22 mA in a low-power operating mode, rising to 51 mA during active cellular data transmission, while Villegas *et al.* (2025) reported a hybrid PLC-ESP32 system achieving roughly 86.72% monthly energy savings at a total component cost under USD 200. Bhaganagare *et al.* (2025a), reviewing voice-processing architectures specifically, found that cloud-based voice processing on the ESP32 introduces 500 to 2,000 ms of latency and fails entirely during internet outages, whereas edge-based on-chip processing achieves sub-100 ms latency and full offline operation, at the cost of a severely restricted vocabulary and higher sensitivity to background noise.

Several limitations recur regardless of the specific application. Weak or degraded Wi-Fi signal is repeatedly identified as the primary source of response lag and command failure, rather than any inherent processing limitation of the chip itself (Ahmed *et al.*, 2024; Villegas *et al.*, 2025). Security is a recurring concern: Ahmed *et al.* (2024) explicitly flag unauthorized access risk and recommend encrypted Wi-Fi connections as a necessary mitigation, while Bhaganagare *et al.* (2025a) note that DIY ESP32 projects commonly omit TLS or secure boot altogether. This is a gap Bhaganagare *et al.* (2025b) address directly in their own design by implementing on-chip MQTT authentication and TLS encryption, demonstrating that the capability exists on this hardware but is inconsistently applied across the literature. A further limitation noted by Aarti *et al.* (2026) is that many ESP32-based automation systems rely on static, pre-configured logic rather than adaptive behavior, limiting their responsiveness to changing conditions without manual reconfiguration.

## 2.5 Energy Sensing

Energy sensing across the reviewed literature converges on two dominant families of approach: Hall-effect current sensors (ACS712, LEM LA-55P) requiring analog signal conditioning and ADC sampling, and pre-integrated multi-parameter modules such as the PZEM-004T that handle voltage, current, and power calculation internally before exposing results over a serial interface. Kashyap and Kashyap (2025) report an ACS712-based system achieving current measurement accuracy up to 99.3% at a 500W load, dropping slightly to 98.5% at 100W, with status-control latency ranging from 0.5 to 0.8 seconds depending on load size. Risfendra *et al.* (2021), using a PZEM-004T sensor instead, report current accuracy of 94.96% under load (rising to 100% at no load) and voltage accuracy averaging 99.42%, comparable to the ACS712 figures despite the different sensing method, though notably lower under active load than the higher-precision PZEM figures reported elsewhere in this review (see Section 2.2.1). Rao *et al.* (2025) took a more rigorous measurement approach, using differential amplifier and DC-offset circuitry to accurately scale the full 230V RMS supply into the Arduino's ADC range rather than assuming a fixed voltage, a design choice the authors explicitly contrast against simpler smart outlet designs that skip this scaling step and consequently introduce measurement error, directly reinforcing the sensor-accuracy tradeoffs already noted in Section 2.2.1.

Beyond basic accuracy, two sources address energy sensing at a more fundamental, sensor-network level rather than the single-plug scale seen elsewhere in this review. Jiang *et al.* (2007) present SPOT, a purpose-built power-metering circuit for wireless sensor network nodes that bypasses conventional ADC-based sampling entirely, instead converting current to a proportional pulse frequency and accumulating energy in a hardware counter with no CPU overhead. This approach achieves a dynamic range of 45,000:1 and microamp-level resolution, substantially exceeding what commodity current sensors in this review report, and highlights a real tradeoff this project's own PZEM-based design accepts: standard ADC sampling requires more frequent CPU polling to capture rapid load transients, at higher power and processing cost than a dedicated hardware-counter approach. Sharma (2023) instead addresses sensing at the network scale, describing a ZigBee-based mesh of 100 energy meters managed by a single gateway, and notes that ZigBee's low power draw and mesh-routing capability extend battery life and reliability compared to higher-power alternatives, though the same paper acknowledges that ZigBee

networks remain susceptible to signal fading, Wi-Fi interference, and increased latency in deeper tree topologies.

A recurring tradeoff across all five sources is between sensing accuracy and system complexity or connectivity dependence. Kashyap and Kashyap (2025) note that ZigBee is favored over Bluetooth and Wi-Fi specifically for energy monitoring because it supports far more concurrent nodes (up to 65,000) at substantially lower power draw, despite Wi-Fi's higher raw throughput. Rao *et al.* (2025) similarly chose a low-power XBee-based mesh network over more capable alternatives to keep overall system cost and latency low. By contrast, Risfendra *et al.* (2021) and most PZEM-based systems reviewed here depend on a continuous Wi-Fi connection and cloud platform for data delivery, with no reported fallback for connectivity loss. This is a limitation directly relevant to this project's own design, which implements local relay state persistence independent of network availability specifically to avoid this class of failure.

Several authors also acknowledge functional limitations shaping future work. Risfendra *et al.* (2021) note their system was designed and tested only for single-phase AC monitoring, with three-phase support left for future iterations, while Rao *et al.* (2025) flag automatic load cutoff as a meaningful risk for devices such as PCs where abrupt power loss can cause data loss, and identify integration with renewable-energy sources and battery state-of-charge monitoring as unaddressed. Taken together, this section reinforces that no single sensing approach is universally superior: the choice between Hall-effect and integrated PZEM-family sensors, and between Wi-Fi and lower-power mesh protocols, is consistently framed in the literature as a deliberate tradeoff between accuracy, cost, power draw, and connectivity resilience rather than a solved, one-size-fits-all decision.

## **2.6 Communication Protocols**

Protocol selection is one of the more heavily benchmarked design decisions in the IoT literature, with MQTT, CoAP, HTTP, AMQP, and OPC UA all evaluated repeatedly under varying network conditions, payload sizes, and reliability requirements.

### **2.6.1 Protocol-Level Comparisons**

Across three independent simulation-based comparisons, CoAP consistently outperforms MQTT and HTTP on raw latency, throughput-per-overhead, and energy efficiency. Silva *et al.* (2021)

found CoAP achieved the lowest completion times in most local and large-scale test configurations, though its performance degraded sharply under long message intervals due to its connection-establishment overhead; MQTT performed competitively at moderate loads but collapsed under high-frequency, large-scale parallel traffic, recording 65.16% packet loss in a one-to-many scenario at 1ms intervals, a failure the authors attribute to MQTT's lack of broker-side message queuing under extreme load. Mishra and Guru (2026) reported a similar pattern under NS-3 simulation: at high stress load, CoAP achieved 0.55ms average latency and 625,017 bps throughput against MQTT's 0.81ms and 139,577 bps, while HTTP suffered complete communication collapse (100% packet loss) under the same conditions. Sifawa *et al.* (2026), simulating a smart-home topology specifically, found CoAP again fastest (30ms latency, 1.9J energy per sensor) but noted that MQTT and HTTP tied for the highest packet delivery ratio (100%) due to their TCP-based reliability mechanisms, concluding explicitly that no single protocol is universally optimal across smart-home application requirements.

Taken together, these findings position MQTT not as the fastest option in raw terms, but as a deliberate trade of some latency and throughput for the persistent, stateful, and reliable connection that continuous appliance monitoring and control require. This is a trade this project accepts by design, prioritizing consistent state delivery over the marginal latency advantage CoAP offers in connectionless, high-frequency sensor scenarios.

### **2.6.2 MQTT Transport and Configuration Tradeoffs**

Within MQTT itself, transport choice and Quality of Service (QoS) level introduce further tradeoffs directly relevant to this project's design. Amirkhanov *et al.* (2025), testing on ESP32 hardware, found MQTT over raw TCP achieved the lowest average latency (290.5ms) but the highest variability (standard deviation of 325.7ms) due to retransmission spikes, while MQTT over WebSocket averaged a slightly higher 342.0ms but with substantially more stable delivery (standard deviation of 193.2ms), making it better suited to browser-based dashboards despite the added encapsulation overhead. Enache *et al.* (2023) confirmed this overhead directly: MQTT over TCP consistently achieved higher throughput and lower connection overhead than MQTT over WebSocket across all tested payload sizes and QoS levels, with WebSocket's only clear advantage being firewall traversal and native web compatibility.

QoS level selection carries its own cost. Hintaw (2025) found that while execution time scales roughly linearly with payload size regardless of QoS, end-to-end latency rises dramatically at higher QoS levels, from 0.66 seconds at QoS 0 to 7.61 seconds at QoS 2 for a 20MB payload, concluding that QoS 0 suits latency-sensitive telemetry while QoS 1/2's acknowledgment handshakes are justified only for mission-critical messages. Eldho *et al.* (2025) reached a compatible conclusion in a smaller-scale ESP32 environmental-monitoring deployment, achieving sub-50ms latencies in practice and noting explicitly that implementing TLS encryption for MQTT is itself a tradeoff, securing the data at the cost of increased latency and power consumption. This finding directly parallels this project's own migration to TLS-secured MQTT: the literature confirms that the small latency and processing cost of encryption is an accepted, documented tradeoff for security, not an unusual or avoidable side effect of the design choice.

### **2.6.3 MQTT Power Efficiency**

Beyond latency, MQTT's persistent-connection model offers a measurable power advantage over HTTP's repeated connection establishment. Jara Ochoa *et al.* (2023), running an 84-hour battery discharge test on cold-chain monitoring hardware, found MQTT QoS 1 consumed 8.33% less power than HTTP, and MQTT QoS 0 consumed 6.03% less, translating to 7 to 10 additional hours of expected battery life on a fixed battery capacity. The authors attribute this directly to HTTP's requirement to open and close a TCP connection on every transmission, against MQTT's single persistent channel. While this project's hardware is mains-powered rather than battery-operated, the same connection-efficiency principle applies to reducing unnecessary network and processing overhead on a resource-constrained microcontroller.

### **2.6.4 Broker Architecture and Scalability**

Broker implementation choice materially affects delivered performance, independent of the protocol itself. Shvaika *et al.* (2025) describe TBMQ's transition from disk-backed to a hybrid in-memory (Redis) and durable-routing (Kafka) architecture, sustaining 1 million messages per second at sub-100ms end-to-end latency and roughly 8ms average publish-acknowledgment latency at that peak load, substantially outperforming disk-based brokers in latency, at the cost of higher memory usage. Mishra, Mishra, and Kertész (2021) stress-tested six broker implementations directly, finding ActiveMQ scaled best under heavy multi-core load while Mosquitto delivered the lowest latency in resource-constrained, single-node environments;

notably, HiveMQ Community Edition recorded the slowest QoS 1 latency among all six brokers tested locally, at 58.48ms compared to Mosquitto's 0.74ms. This finding should be read carefully rather than applied directly to this project: the tested "HiveMQ CE" is a self-hosted, open-source broker distinct from HiveMQ Cloud's managed service, which this project actually uses, and the two are not guaranteed to share performance characteristics. It does, however, underscore a broader point supported across this section: broker choice is not a neutral implementation detail, and empirically measuring one's own deployed broker's latency, rather than assuming a vendor's general reputation, is the methodologically sound approach the reviewed literature consistently takes.

Across this section, the reviewed literature converges on a consistent picture: MQTT is not the fastest protocol in raw benchmarks, but its persistent connection model offers meaningful power and reliability advantages over HTTP, its QoS levels provide a tunable reliability-latency tradeoff, and its security overhead via TLS is a well-documented, acceptable cost rather than an anomaly. Collectively, this supports MQTT-over-TLS as a defensible, literature-consistent protocol choice for this project's device-to-backend communication.

## **2.7 Backend/Remote-Control Architecture**

Backend architecture choices across the reviewed literature fall into three broad categories: MQTT broker-based systems, serverless cloud-database architectures, and direct REST API implementations, each carrying distinct implications for reliability, scalability, and security.

Two systems adopt an MQTT broker as the central communication hub, closely matching this project's own backend approach. Sayanna *et al.* (2026) route commands through a cloud-hosted broker (Mosquitto or CloudMQTT), with the ESP8266 subscribing to device-specific topics and publishing status confirmations back to the broker for real-time user feedback; the authors report that this publish-subscribe model minimizes network overhead and sustains stable communication under unstable network conditions compared to polling-based HTTP. Kolanur *et al.* (2021) extend this pattern with a Raspberry Pi acting as both the MQTT server and the host for the Domoticz dashboard platform, but distinguish their design through deliberate control-path redundancy: rather than relying solely on Wi-Fi/MQTT, appliances remain controllable via Bluetooth, infrared, physical keypads, or wall-mounted touch panels, so that a failure in one control path does not leave the appliance completely unreachable. This directly parallels this project's own local-fallback

design principle. Kolanur *et al.* explicitly frame this multi-path approach as protecting the end user from being restricted to a single point of failure, the same reasoning underlying this project's local button-triggered relay control during MQTT disconnection.

A different architectural pattern appears in S and K (2023), who adopt a fully serverless model built on Google Firebase's Realtime Database rather than a self-hosted broker or server. Commands are written directly as JSON state changes, which Firebase automatically synchronizes to the connected NodeMCU, eliminating the need to provision, host, or pay for dedicated server infrastructure. The authors report this approach as highly cost-effective, citing a prototype cost under \$5,000, and highlight that it removes the recurring costs associated with SMS-based or locally-hosted alternatives. This architecture trades the greater control and customization of a self-managed broker for reduced operational overhead, a reasonable choice for a smaller-scale deployment, though it introduces a dependency on a third-party platform's availability and pricing model that a self-hosted or managed-broker approach avoids.

Oo, Lai, and Moe (2022) instead reject the publish-subscribe model entirely in favor of a REST API architecture, hosting a lightweight web server directly on the microcontroller using the aREST library and issuing standard HTTP commands from a browser-based interface via PHP and cURL. The authors report a specific practical advantage of this approach: the device firmware can be flashed once and left unchanged, with all subsequent interface changes made entirely in browser-side code rather than requiring reflashing. This is a meaningful development-workflow benefit, though one that comes at the cost of HTTP's inherently higher per-request overhead and connection-teardown cost relative to MQTT's persistent-connection model, as established in Section 2.6. Notably, Oo, Lai, and Moe (2022) explicitly acknowledge that their prototype lacks any network security protection, identifying secure login and credentialed access as necessary future work; this is a gap directly contrasted by Sayanna *et al.* (2026), whose broker-based design explicitly implements encrypted MQTT communication and per-device credentials from the outset.

Across all four systems, command flow follows a consistent structural pattern regardless of backend choice: user input triggers a state change, that state change propagates to the microcontroller, and the microcontroller drives a relay to physically switch the appliance. The meaningful architectural differences are concentrated in how reliably and securely that propagation

occurs under real-world network conditions. This reinforces the design rationale established in Section 2.6: an MQTT broker-based backend, secured with authentication and TLS, offers a more literature-consistent balance of reliability and security than either a fully serverless third-party dependency or an unsecured REST implementation, while the redundant local-control principle demonstrated by Kolanur *et al.* (2021) directly supports this project's own approach to maintaining appliance control during connectivity loss.

## **2.8 IoT Security**

Security is one of the most heavily documented weaknesses in the IoT literature reviewed for this project, spanning network communication, firmware, hardware, and operating-system-level vulnerabilities.

### **2.8.1 Scale and Severity of the Problem**

Multiple sources quantify just how widespread IoT security weaknesses are. Kabir *et al.* (2023), synthesizing 70 published cybersecurity works, report that 98% of all IoT data traffic is transmitted over unencrypted channels, and that up to 57% of connected IoT devices are vulnerable to moderate-to-high severity attacks. Rajasekar and Rajkumar (2023) confirm this at internet scale through a direct Shodan-based scan of 210,125 publicly accessible IoT devices, finding 13.52% vulnerable overall, with 46.22% of vulnerable devices rated high-risk and 12.75% rated critical using the Octave Allegro risk methodology; among the specific findings, 78.31% of insecure devices ran unencrypted Telnet, and over 16,900 systems were still using default vendor-supplied credentials. Kim and Yoo (2022), analyzing the 2019 CVE database directly, found that IoT-specific vulnerabilities were disproportionately severe compared to vulnerabilities generally, 41.80% rated high-severity versus 25.64% across the full CVE database that year, and that 84.8% of IoT vulnerabilities were hardware-dependent rather than software-dependent, underscoring that security weaknesses in this domain are frequently structural rather than easily patchable.

### **2.8.2 Network and Communication Vulnerabilities**

Unencrypted or poorly secured communication is a recurring theme directly relevant to this project's own broker-security decisions. Malik and P. (2023) categorize insecure communication protocols, including unencrypted HTTP and MQTT, as a primary network-layer vulnerability enabling packet sniffing and man-in-the-middle attacks, alongside DDoS and data-breach risks, citing the 2016 Mirai botnet's exploitation of millions of vulnerable devices as a real-world

consequence of this class of weakness. Murat *et al.* (2024) demonstrated this concretely against smart plug hardware specifically: they extracted a complete unencrypted firmware image from a Gosund SP111 smart plug and found plaintext JSON containing the device's Wi-Fi credentials and authorization key stored directly in flash memory, and separately sniffed the full Zigbee network key of a Gosund IG019 smart plug because its coordinator used a default, publicly known transport key. These findings are a direct, device-level validation of the general MQTT security concerns addressed in Section 2.6 and reinforce the rationale behind this project's migration from an unauthenticated public broker to a TLS-secured, credentialed HiveMQ Cloud instance. This is the exact category of exposure Murat *et al.* document as exploitable in commodity smart plug hardware.

### **2.8.3 Firmware and Hardware-Level Vulnerabilities**

Beyond network communication, firmware and low-level code quality represent a distinct vulnerability surface. Al-Boghdady *et al.* (2021) found a strong correlation between codebase size and security error density across four popular IoT operating systems, with the worst-performing system (Contiki) showing roughly six times the error density of the best (RIOT), attributing the difference primarily to the presence or absence of enforced repository security policies during development. Bakhshi *et al.* (2024) broaden this to firmware generally, citing a large-scale analysis of 32,000 firmware images that uncovered 693 vulnerabilities including 38 zero-day exploits, and noting that 98% of network-based attacks observed in 2023 honeypot data specifically targeted the Telnet interface, consistent with the Telnet exposure figures reported by Rajasekar and Rajkumar (2023). At the most granular hardware level, Ruminot *et al.* (2023) demonstrated that resource-constrained microcontrollers such as the ATmega328P are vulnerable to low-cost voltage fault injection attacks capable of bypassing password authentication checks with roughly a 50% success rate per attempt using equipment costing only a few dollars, illustrating that even physically-present, low-budget attacks pose a credible threat to embedded authentication logic.

### **2.8.4 Recommended Mitigations**

Despite differing focus areas, the reviewed literature converges on a consistent set of countermeasures. TLS/DTLS encryption for data in transit and AES-256 for data at rest are recommended repeatedly (Kabir *et al.*, 2023; Almani and Furnell, 2025; Malik and P., 2023), alongside network segmentation to isolate IoT devices from broader infrastructure (Rajasekar and

Rajkumar, 2023; Murat *et al.*, 2024) and the elimination of default credentials in favor of unique, mandatory per-device authentication (Almani and Furnell, 2025; Rajasekar and Rajkumar, 2023). Several sources also emphasize secure-by-design development practices, enforced repository security policies, secure boot, and restricting or removing physical debugging interfaces before production (Al-Boghdady *et al.*, 2021; Murat *et al.*, 2024), rather than treating security as a feature added after core functionality is complete.

### **2.8.5 Relevance to This Project**

Taken together, this section's findings directly validate two of this project's core security interventions. The unauthenticated, publicly accessible broker.hivemq.com instance used during earlier development corresponds precisely to the insecure-network-service pattern Rajasekar and Rajkumar (2023) found exploited at scale, and its replacement with a TLS-secured, credentialed private broker addresses the encryption and authentication gaps identified across nearly every source in this section. Separately, the discovery and removal of an unauthenticated backend endpoint exposing the full user database reflects exactly the kind of access-control failure Kabir *et al.* (2023) and Almani and Furnell (2025) identify as a common, avoidable consequence of prioritizing functionality over security during development, reinforcing that these were not isolated implementation mistakes, but instances of a well-documented, industry-wide pattern this project's design choices directly and demonstrably addressed.

## **2.9 Research Gap and Summary**

This chapter has reviewed literature across eight interconnected domains relevant to the design of an IoT-based smart WiFi plug: foundational IoT and smart-home concepts, existing smart plug and broader home-automation systems, ESP32-family microcontroller platforms, energy-sensing hardware, communication protocols, backend and remote-control architectures, and IoT security. Across this body of work, several consistent patterns emerge that together define the specific gap this project addresses.

This project's specific contribution is a smart WiFi plug system that combines the well-established sensing and control capabilities documented throughout this chapter with two elements the reviewed literature shows are frequently under-addressed in combination: TLS-secured, authenticated device-to-broker communication, and explicit local state persistence and control-path redundancy during connectivity loss, implemented within a single-developer project timeline,

without the added architectural complexity that comparably secure systems in the reviewed literature typically require. This positions the project not as introducing a fundamentally new technical capability, but as demonstrating that a documented, industry-recognized security and reliability gap in this specific class of device can be closed without disproportionate added cost or complexity.

## CHAPTER THREE

### METHODOLOGY

#### 3.1 Development Methodology

This project was developed using an iterative, prototype-driven methodology rather than a single-pass linear process. Each development cycle involved implementing a discrete feature or fix, testing it against the running hardware and back-end, and refining the design based on issues surfaced during that testing before moving to the next cycle. This approach was chosen deliberately over a strict Waterfall model because embedded IoT systems of this kind involve tight coupling between firmware, network communication, and back-end behavior, meaning design assumptions made early frequently needed revision only after being tested against real hardware and network conditions, rather than being fully specifiable upfront.

Development proceeded across three broad phases. The first phase established core functionality: firmware capable of relay switching and telemetry publishing, a back-end capable of receiving and storing that telemetry, and a mobile application capable of displaying it. The second phase focused on feature expansion, including user profile management, device timers, and analytics. The third phase, most relevant to this report's account of the completed prototype, focused on refactoring, systematic code review, and closing two specific pre-launch priorities repeatedly identified across earlier development cycles: MQTT broker security and keep-alive configuration.

Testing throughout development combined manual functional verification with targeted code review passes that surfaced and resolved several defects prior to hardware integration, including a relay-state synchronization bug between remote and local control paths and an unauthenticated back-end endpoint exposing user data. This methodology is reflected in the system design decisions presented throughout the remainder of this chapter.

#### 3.2 System Overview

The Smart WiFi Plug system is designed as a four-layer distributed architecture in which a Wi-Fi-enabled smart plug, a cloud-hosted MQTT broker, a FastAPI backend server, and a React Native mobile application each fulfil a distinct role. The smart plug, built around the ESP32-C3 microcontroller and the PZEM-004T energy monitoring module, forms the physical layer: it continuously reads electrical measurements, publishes telemetry to the broker over MQTT, and responds to control commands received from the same broker. The backend server subscribes to

the broker, persists incoming telemetry into a PostgreSQL database, enforces configurable energy limits, and exposes a REST API that the mobile application uses for historical data and account management. The mobile application communicates with the broker directly for real-time commands and live data, and with the backend over HTTPS for historical records, timer management, and account operations. A visual representation of the system architecture is represented in Figure 3.1.

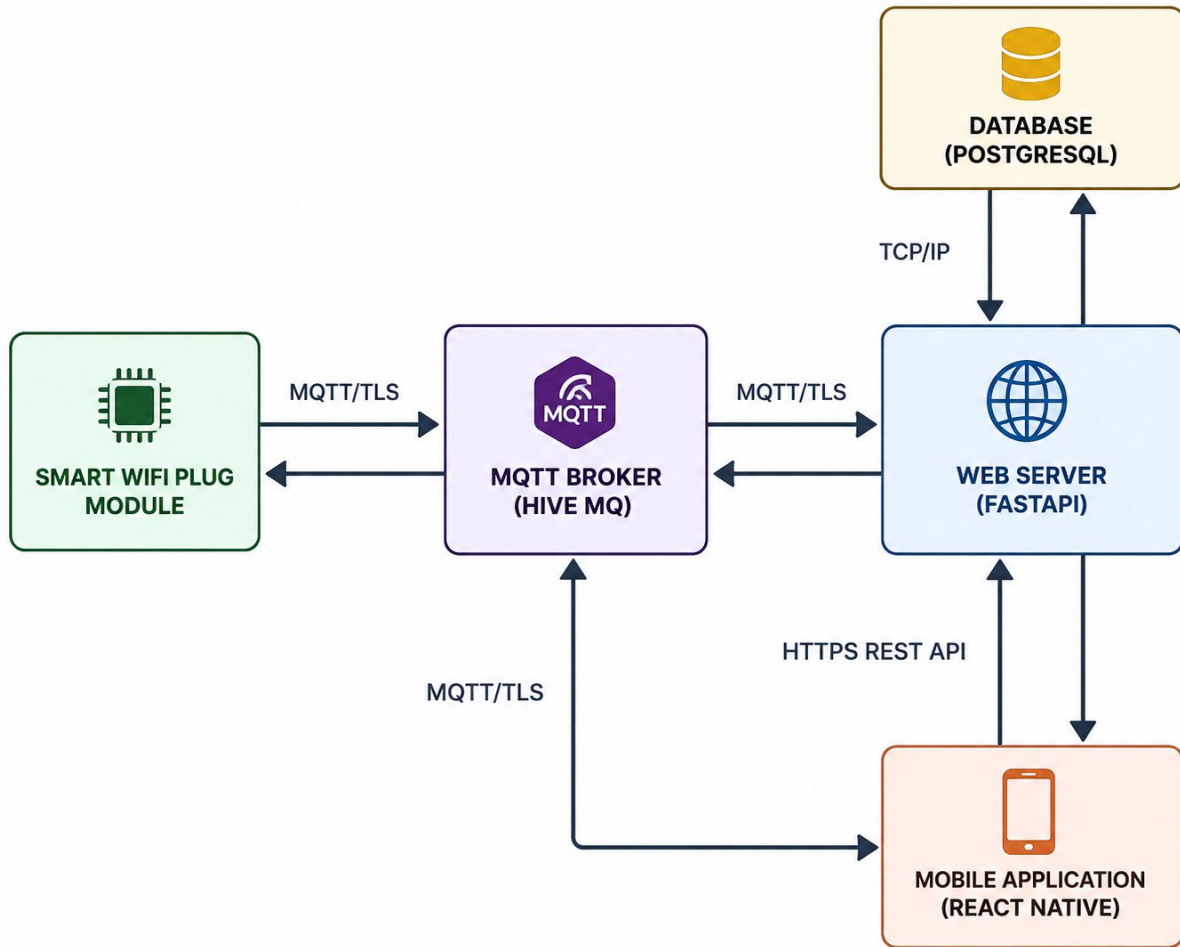


Figure 3.1: Overall system architecture

### 3.3 Hardware Design and Implementation

#### 3.3.1 Component Selection

The hardware design was guided by three constraints: electrical safety at mains voltage, physical compactness to fit a standard single-gang back box, and cost-effectiveness for a prototype, directly

fulfilling Objective 1's requirement for an ESP32 and PZEM-004T based hardware architecture. The selected components with their specifications and selection rationale are described in Table 3.1. The component images are shown in Figures 3.2 to 3.11.

Table 3.1: Table of components used, specification, justification, and Image.

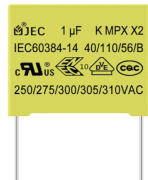
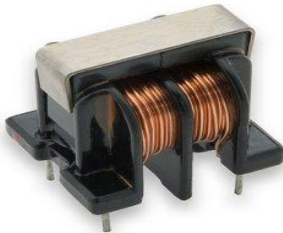
| Component              | Specification                                              | Justification                                                                                                        | Image                                                                                 |
|------------------------|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| ESP32-C3 Mini          | Dual-core 240 MHz, Wi-Fi 802.11 b/g/n, 520 KB SRAM         | Mature ecosystem, native ESP-IDF support, integrated Wi-Fi eliminates the external module                            |    |
| PZEM-004T v3.0 (100A)  | Voltage: 80-260V, Current: 0-100A, UART/Modbus RTU, TTL 5V | Comprehensive measurement (voltage, current, power, energy, frequency, PF), non-invasive CT clamp, widely documented |    |
| HLK-PM05               | Input: 100-240 VAC, Output: 5 VDC 1000 mA                  | Compact encapsulated module, certified, no external components required for basic operation                          |  |
| 1-Channel Relay Module | Coil 5V, contacts 10A 250VAC, optically isolated           | Optocoupler isolates microcontroller logic from relay coil transients; plug-in module simplifies assembly            |  |

Figure 3.2: ESP32-C3 Mini

Figure 3.3: PZEM-004T Module

Figure 3.4: HLK-PM05

Figure 3.5: 1-Channel Relay Module

| Component                             | Specification                                      | Justification                                                                                          | Image                                                                                                                       |
|---------------------------------------|----------------------------------------------------|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Current Transformer (100A split-core) | 100A rated, compatible with PZEM-004T 100A version | Non-invasive measurement; avoids breaking the live conductor                                           |  <p>Figure 3.6: Current Transformer</p>  |
| Fuse                                  | Ceramic cartridge, 13A 5x20mm, slow-blow           | Protects entire circuit from catastrophic overcurrent; slow-blow tolerates inrush current at switch-on |  <p>Figure 3.7: Fuse</p>                 |
| Varistor (S10K230)                    | 230V rated, 10mm disc                              | Absorbs voltage transients and surge events on the mains input                                         |  <p>Figure 3.8: Varistor</p>            |
| X2 Safety Capacitor (0.1uF 275VAC)    | Class X2, polypropylene film                       | EMC line filtering; fail-safe behaviour, opens rather than shorts on failure                           |  <p>Figure 3.9: Class X2 Capacitor</p> |
| Common Mode Choke (2mH 10A)           | 2mH inductance, 10A current rating                 | Suppresses common-mode EMI generated by the switching power supply                                     |  <p>Figure 3.10: Common Mode Choke</p> |

| Component                    | Specification                     | Justification                                                                               | Image                                                                               |
|------------------------------|-----------------------------------|---------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Output Capacitor (100uF 25V) | Electrolytic, low - ESR preferred | Smooths the 5V DC rail output; reduces ripple reaching the microcontroller and relay module |  |

Figure 3.11: Output Capacitor

### 3.3.2 Power Supply Circuit

The power supply circuit converts the 230V AC mains input to the 5V DC rail required by the relay module and the PZEM-004T TTL interface, and to the 3.3V supply required by the ESP32-C3 core, regulated by the DevKit's onboard linear regulator. The Live conductor passes through a slow-blow fuse, a metal oxide varistor (S10K230) for surge suppression, a Class X2 safety capacitor for high-frequency noise filtering, and a common-mode choke to attenuate electromagnetic interference from the HLK-PM05's internal switching regulator, before reaching the HLK-PM05 module itself. The module's DC output is decoupled with a 100uF electrolytic capacitor providing bulk charge storage and high-frequency decoupling for the 5V rail. Together with the optically isolated relay described in Section 3.4.5, these protection elements fulfil Objective 3's requirement for appropriate isolation and protection mechanisms in the load-switching circuitry. A schematic of the system is shown in Figure 3.12.

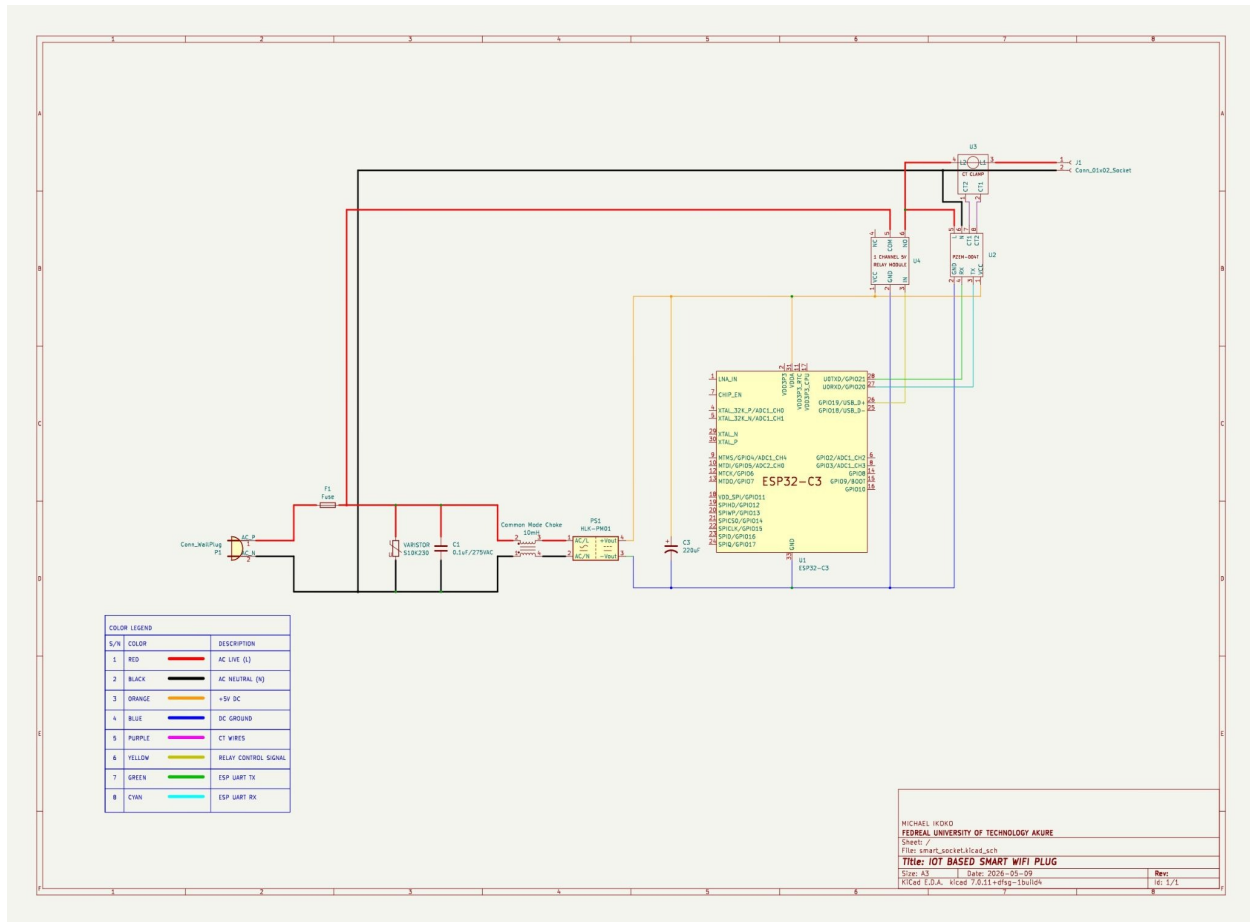


Figure 3.12: Circuit Schematic

### 3.4 Measurement and Control Subsystem

The measurement and control subsystem integrates the electrical measurement, data acquisition, and load-switching functions of the smart plug. It comprises four principal functional elements: voltage sensing, current sensing, electrical parameter calculation, and relay-based load control. The PZEM-004T-100A performs the measurement and calculation of electrical parameters, while the ESP32-C3 receives the measurements through a TTL-level UART interface and controls the relay to switch the connected load. The operation of these elements is described below.

#### 3.4.1 Voltage Sensing

The PZEM-004T connects directly to the AC mains via its L and N terminals, wired in parallel with the load. Internally, the module uses a precision resistor-divider network to step the 80-260V mains voltage down to a level safe for sampling by its internal analog-to-digital converter, while maintaining electrical isolation between the mains-side circuitry and the low-voltage TTL

communication side (PZEM-004T V3.0 User Manual, n.d.; CircuitSchools Staff, 2026). The metering IC samples the divided voltage waveform at high frequency and computes the true root-mean-square (RMS) value in hardware, rather than a simple peak or average reading, which is essential given that mains voltage in a real installation is rarely perfectly sinusoidal:

$$V_{r.m.s} = \sqrt{\frac{1}{T} \int_0^T V(t)^2 dt} \quad (3.1)$$

where  $v(t)$  is the instantaneous voltage waveform and  $T$  is one complete AC cycle.

### 3.4.2 Current Sensing (Toroidal CT Transformer)

Current is measured non-invasively using a clamp-on current transformer (CT), a critical design choice since it allows measurement without breaking or exposing the live conductor, reducing installation risk. The CT consists of a toroidal (ring-shaped) ferrite core that clamps around the live conductor; the conductor itself acts as a single-turn primary winding passing through the core, while a secondary winding of many turns is wound around the core body. When alternating current flows through the live conductor, it generates a changing magnetic flux in the core, which by Faraday's law of electromagnetic induction induces a proportional alternating current in the secondary winding. The relationship between primary and secondary current follows the transformer turns-ratio (ampere-turn balance) equation:

$$N_p \times I_p = N_s \times I_s \quad (3.2)$$

Since the live conductor passes through the core only once,  $N_p = 1$ , so the secondary current is:

$$I_s = \frac{I_p}{N_s} \quad (3.3)$$

where  $I_p$  is the true load current and  $N_s$  is the number of turns on the secondary winding. This small secondary current is converted into a proportional voltage internally and fed to the PZEM-004T's metering IC, which computes true RMS current in the same manner as voltage:

$$I_{r.m.s} = \sqrt{\frac{1}{T} \int_0^T i(t)^2 dt} \quad (3.4)$$

A critical installation rule follows directly from this principle: the CT must be clamped around the live conductor only. If both live and neutral conductors pass through the core, their opposing magnetic fields cancel and the measured current reads zero (CircuitSchools Staff, 2026). This is why, in Section 3.4.2's circuit description, the CT clamp is explicitly installed on the Live wire on the load side of the relay, and not around both conductors of the appliance cord.

### 3.4.3 Power, Power Factor, and Energy Calculation

With true RMS voltage and current established, the module derives the remaining electrical parameters. Apparent power is the simple product of RMS voltage and current:

$$S = V_{r.m.s} \times I_{r.m.s} \quad (3.5)$$

Real (active) power, the quantity actually billed by utilities and displayed as the primary reading, accounts for the phase difference between the voltage and current waveforms:

$$P = V_{r.m.s} \times I_{r.m.s} \times \cos(\phi) = S \times P.F \quad (3.6)$$

where phi is the phase angle between the voltage and current waveforms. Power factor is then defined as the ratio of real to apparent power:

$$PF = \frac{P}{S} = \cos(\phi) \quad (3.7)$$

A power factor near 1.0 indicates a purely resistive load (such as a heater), while a lower power factor indicates a more reactive, typically inductive, load such as a motor or compressor (PZEM-

004T V3.0 User Manual, n.d.). Accumulated energy is not measured directly but derived by integrating instantaneous power over time:

$$E = \int_0^T P(t) dt \quad (3.8)$$

Because the module samples power at discrete intervals rather than continuously, this integral is approximated internally as a summation of incremental energy contributions:

$$E = \sum_1^n (P_i \times \Delta t_i) \quad (3.9)$$

This accumulated energy value is stored in a 32-bit register pair (registers 0x0005-0x0006) and increases monotonically; per the module's communication protocol, it can only be cleared using the dedicated reset-energy command (function code 0x42), not by any read operation (PZEM-004T V3.0 User Manual, n.d.). This is precisely why the backend never attempts to reset the device's internal counter and instead computes period consumption by subtracting successive readings:

$$kWh_{consumed} = Energy_{last} - Energy_{first} \quad (3.10)$$

and, using the user's configured billing rate, estimated cost:

$$cost = kWh_{consumed} \times billing\_rate \quad (3.11)$$

The PZEM-004T communicates these six derived parameters, voltage, current, power, energy, frequency, and power factor, to the ESP32-C3 via a TTL-level UART interface implementing the Modbus RTU protocol, at 9600 baud with 8 data bits, 1 stop bit, and no parity (PZEM-004T V3.0 User Manual, n.d.), drawing its 5V communication-side supply from the HLK-PM01 output rail.

### 3.4.4 PZEM-004T Communication Protocol

The physical layer is a UART interface exposing GND, TX, RX, and 5V pins, operating at 9600 baud with 8 data bits, 1 stop bit, and no parity (commonly written as 8N1).

The application layer uses the Modbus RTU protocol, supporting function codes 0x03 (Read Holding Register), 0x04 (Read Input Register), 0x06 (Write Single Register), 0x41 (Calibration, factory use only), and 0x42 (Reset Energy). Slave addresses range from 0x01 to 0xF7, with 0x00 reserved as a broadcast address and 0xF8 reserved as a general address for single-slave setups such as this project's. Table 3.2 lists the register map used to retrieve measurement results via function code 0x04.

Table 3.2: PZEM-004T Modbus register map for measurement results (PZEM-004T V3.0 User Manual, n.d.)

| Register Address | Description                                      | Resolution |
|------------------|--------------------------------------------------|------------|
| 0x0000           | Voltage                                          | 0.1 V      |
| 0x0001-0x0002    | Current (low/high 16 bits)                       | 0.001 A    |
| 0x0003-0x0004    | Power (low/high 16 bits)                         | 0.1 W      |
| 0x0005-0x0006    | Energy (low/high 16 bits)                        | 1 Wh       |
| 0x0007           | Frequency                                        | 0.1 Hz     |
| 0x0008           | Power factor                                     | 0.01       |
| 0x0009           | Alarm status (0xFFFF = alarm, 0x0000 = no alarm) | -          |

### 3.4.5 Relay Control

The relay module provides load switching and completes the requirement of Objective 3. Its control side connects to GPIO pin 5 on the ESP32-C3, with its optocoupler electrically isolating this low-voltage control signal from the relay coil and, by extension, from the switched mains circuit. The relay is wired normally-open, so the load is disconnected by default when no power is applied to the coil, the safe default state in the event of firmware crash or unexpected reset.

## **3.5 Firmware Architecture**

### **3.5.1 Development Environment**

The firmware is developed using the Espressif IoT Development Framework (ESP-IDF), built on FreeRTOS, which provides preemptive multitasking through a task-based execution model. Each independently executing unit of work, PZEM reading, MQTT publishing, relay control, and timer evaluation, runs as a FreeRTOS task with an assigned stack size, priority, and core affinity, with inter-task communication handled through FreeRTOS queues and mutexes. The ESP-IDF component model organizes the PZEM-004T driver as a self-contained, reusable component, separating the sensor interface from application logic. MQTT topic strings are generated once via a lazy-initialization pattern (`mqtt_topics_init()`) using the device's derived unique ID, then cached and returned by simple getter functions, avoiding repeated string formatting on every publish.

### **3.5.2 Device Operating Modes**

The firmware defines two operating modes based on the presence of provisioning data in NVS (non-volatile storage), a key-value flash partition that survives power cycles and resets. On boot, the firmware checks NVS for Wi-Fi credentials, a device name, and MQTT broker configuration. If any are absent, the device enters pre-configuration mode, starting a Wi-Fi SoftAP (SSID Smartplug-4E13F0, derived from the device's MAC address) and a lightweight HTTP server for provisioning. The relay remains toggleable by the physical button during this mode, so the socket functions as an ordinary outlet even before registration. Once valid credentials are received, they are written to NVS and the device restarts into post-configuration mode: connecting to Wi-Fi, synchronizing time via SNTP, establishing a TLS-encrypted MQTT connection, publishing online status, subscribing to command topics, and starting all operational tasks. A flowchart showing the device operating mode is shown in Figure 3.13.

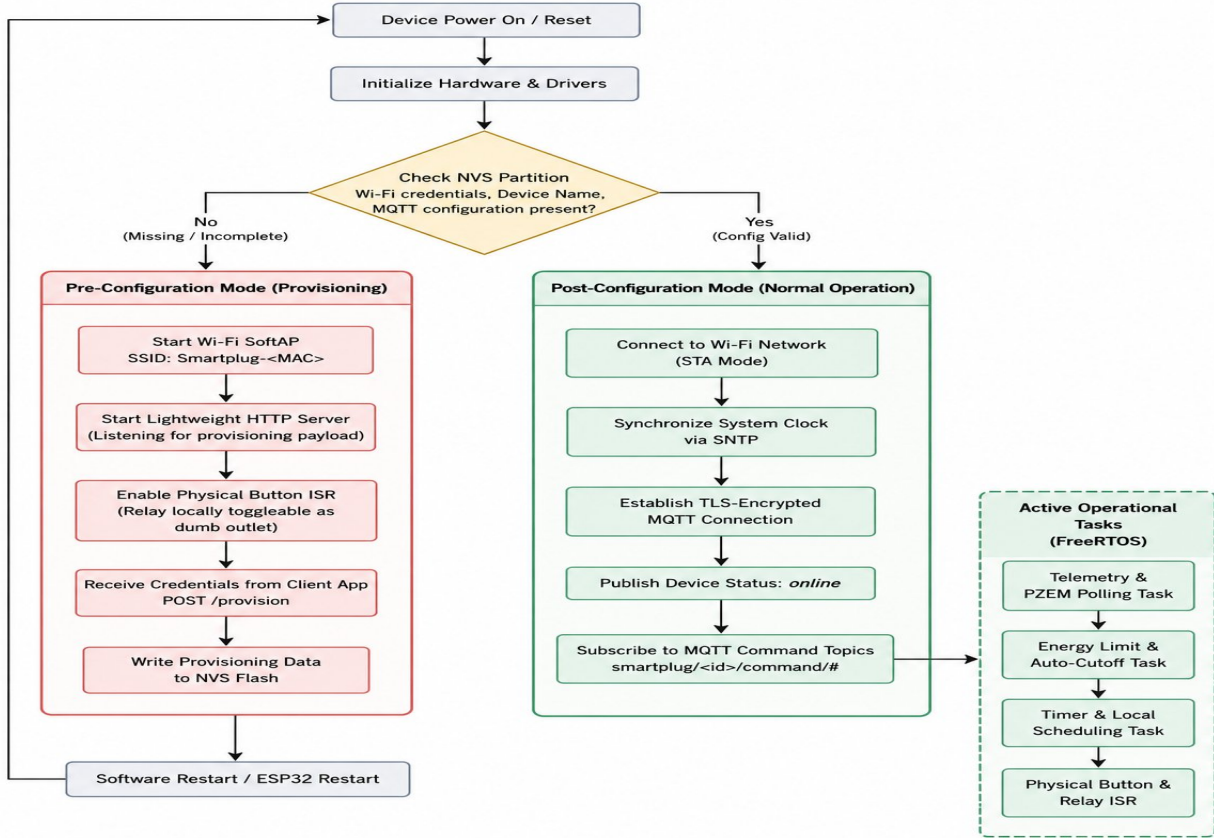


Figure 3.13: Operating Modes

### 3.6 Communication Protocol

#### 3.6.1 Protocol Selection

MQTT was selected as the primary communication protocol between the smart plug, backend, and mobile application, for the reasons established in Section 2.6, and fulfils Objective 2's requirement for Wi-Fi and MQTT-based data transmission: its persistent, publish-subscribe model decouples publishers from subscribers, so the smart plug does not need to know whether the mobile application is currently open, and the application receives real-time updates without polling.

#### 3.6.2 MQTT Topic Structure

MQTT defines three Quality of Service (QoS) levels that determine message delivery reliability. Level 0 provides at most once delivery with no re-transmission, level 1 provides at-least-once delivery with possible duplicates, and level 2 provides exactly-once delivery through a higher-overhead handshake. In this system, level 0 is used for telemetry, while level 1 is used for commands and important state or configuration messages.

The actual implemented topic structure, drawn directly from the firmware's topic initialization and the mobile client's subscription logic, is described by Table 3.3 below.

Table 3.3: MQTT Topic Structure

| Topic                                    | Publisher | Subscriber   | QoS | Purpose                                                                 |
|------------------------------------------|-----------|--------------|-----|-------------------------------------------------------------------------|
| smartplug/{device_id}/status             | Device    | Backend, App | 1   | Online status broker publishes offline via LWT on ungraceful disconnect |
| smartplug/{device_id}/telemetry          | Device    | Backend, App | 0   | Live voltage, current, power, energy, frequency, PF, and relay readings |
| smartplug/{device_id}/relay/state        | Device    | Backend, App | 1   | Current relay state and trigger source (app ,button, boot)              |
| smartplug/{device_id}/relay/command      | App       | Device       | 1   | Relay control commands: ON, OFF, TOGGLE                                 |
| smartplug/{device_id}/wifi/command       | App       | Device       | 1   | New Wi-Fi credentials for a network change                              |
| smartplug/{device_id}/wifi/result        | Device    | App          | 1   | One-shot result of a Wi-Fi change attempt                               |
| smartplug/{device_id}/be-online-status   | Backend   | App          | 1   | Backend computed online and offline status                              |
| smartplug/{device_id}/be-timer-lock      | Backend   | App          | 1   | Whether an energy-limit cutoff overrides timer control                  |
| smartplug/{device_id}/be-daily-summary   | Backend   | App          | 1   | Backend computed current day energy summary                             |
| smartplug/{device_id}/be-monthly-summary | Backend   | App          | 1   | Backend computed current month energy summary                           |
| smartplug/{device_id}/energy-event       | Backend   | App          | 1   | Notification that a daily and monthly energy limit is hit.              |

The representative JSON payloads for these topics are shown in Table 3.4 below.

Table 3.4: Payload of MQTT Topics

| Topic                                      | Example Payload                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| smartplug/{device_id}/<br>telemetry        | { "v":220.1,"i":4.53,"p":997.2,"e":1.245,"f":50.0,"pf":0.99,"relay":1,<br>"ts":1735000000,"rssi":-52}                                                                                                                                                                                                                                                                                                                                                                   |
| smartplug/{device_id}/<br>relay/state      | { "state":"ON","source":"app","ts":1735000000}                                                                                                                                                                                                                                                                                                                                                                                                                          |
| smartplug/{device_id}/<br>relay/command    | { "cmd":"TOGGLE"}                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| smartplug/{device_id}/<br>wifi/command     | { "ssid":"HomeNetwork","password":"..."}                                                                                                                                                                                                                                                                                                                                                                                                                                |
| smartplug/{device_id}/<br>wifi/result      | { "status":"success","ts":1735000000}                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| smartplug/{device_id}/<br>be-timer-lock    | { "locked":true,"reason":"daily_limit_reached","locked_at":"2026-<br>01-15T10:00:00Z"}                                                                                                                                                                                                                                                                                                                                                                                  |
| smartplug/{device_id}/<br>be-online        | { "is_online":true}                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| smartplug/{device_id}/<br>be-daily-summary | { "device_id":"esp32-smartplug-001",<br>"date":"2026-07-22", "energy_first":0.0,<br>"energy_first_timestamp":"2026-07-22T02:08:49+01:00",<br>"energy_last":0.3055555555555556,<br>"energy_last_timestamp":"2026-07-22T02:39:20Z",<br>"kwh_consumed":0.3056,"peak_power":110.0,<br>"peak_power_timestamp":"2026-07-22T02:09:09+01:00",<br>"updated_at":"2026-07-22T03:39:11.125120+01:00",<br>"created_at":"2026-07-22T02:08:49.475396+01:00",<br>"estimated_cost":6402} |
| smartplug/{device_id}/                     | { "device_id":"esp32-smartplug-001", "month":"2026-07",                                                                                                                                                                                                                                                                                                                                                                                                                 |

| Topic                                  | Example Payload                                                                                                                                                                                    |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| be-monthly-summary                     | "kwh_consumed":0.9756, "estimated_cost":20438}                                                                                                                                                     |
| smartplug/{device_id}/<br>energy-event | { "id": 393, "event_type": "daily_limit_reached",<br>"period": "daily", "period_key": "2026-07-21",<br>"kwh_at_event":0.076, "limit_kwh":0.075, "created_at": "2026-07-21T10:37:04.825154+01:00" } |

### 3.7 Cloud Infrastructure, Backend and Mobile Application

#### 3.7.1 MQTT Broker

HiveMQ Cloud is used as the managed MQTT broker, providing TLS encryption on port 8883(WebSocket) and username/password authentication. Using a managed broker eliminates the need to provision and maintain broker infrastructure, letting development effort concentrate on the firmware, backend, and mobile layers.

#### 3.7.2 Backend Server

The backend server is implemented in Python using FastAPI and SQLAlchemy, backed by a PostgreSQL database. It performs two functions: subscribing to the MQTT broker to persist incoming telemetry and compute derived summaries, and exposing a REST API over HTTPS that the mobile application uses for authentication, device management, timer configuration, and historical data retrieval. Rather than using a single wildcard subscription across all devices, the server subscribes to each registered device's telemetry topic individually as devices are registered or come online, inserting each reading into the telemetry table and updating the corresponding day's row in the daily summary table.

#### 3.7.3 Database Design

The database schema, drawn from the actual SQLAlchemy definitions, comprises eight database tables, which is shown in Table 3.5.

Table 3.5: Database tables, columns and purpose.

| Table | Key Columns                                                   | Purpose                                                           |
|-------|---------------------------------------------------------------|-------------------------------------------------------------------|
| users | id, full_name, email, password_hash<br>(bcrypt), billing_rate | Registered accounts; password<br>never stored in plaintext and is |

| Table             | Key Columns                                                                                                                                                            | Purpose                                                                                        |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
|                   |                                                                                                                                                                        | hashed and encrypted                                                                           |
| devices           | id, device_id (unique), user_id (FK), relay_state, is_enabled, is_online, daily_limit_kwh, monthly_limit_kwh, auto_cutoff_enabled, cutoff_reason, cutoff_at, last_seen | Registered smart plugs; tracks live state and energy-limit cutoff status independently         |
| device_timers     | id, device_id (FK), time, action, is_enabled, last_triggered_date                                                                                                      | Recurring daily ON/OFF schedule entries; last_triggered_date prevents repeated same-day firing |
| telemetry         | id, device_id (FK), timestamp, voltage, current, power, energy, frequency, pf, relay, rssi                                                                             | One row per MQTT telemetry message; energy is a cumulative counter that never resets           |
| energy_daily      | id, device_id (FK), date, energy_first, energy_last, kwh_consumed, peak_power                                                                                          | One row per device per day; consumption computed without scanning the full telemetry table     |
| energy_events     | id, device_id (FK), user_id (FK), event_type, period, period_key, kwh_at_event, limit_kwh, is_read                                                                     | Records limit-reached events; unique constraint prevents duplicate firing per period           |
| refresh_token     | id, token (unique), user_id (FK), revoked, expires_at                                                                                                                  | Supports token rotation, immediate logout revocation, and reuse detection                      |
| otp_codes         | otp_hash, purpose, attempts, consumed                                                                                                                                  | OTP codes stored hashed, capped at 5 attempts                                                  |
| used_reset_tokens | user_id, jti                                                                                                                                                           | Records the used reset-token IDs to make it single-use                                         |

### 3.7.4 Mobile Application Screens

The mobile application is built using Expo Router and React Native, with Gluestack UI and Tailwind CSS for component styling, Zustand for local state management, and React Query for server-state caching and synchronization. The application comprises of 19 screens spanning authentication, device onboarding, and core monitoring and control functionality, described in Table 3.6.

Table 3.6: Mobile Application Screens

| Category       | Screen             | Purpose                                                    |
|----------------|--------------------|------------------------------------------------------------|
| Authentication | Welcome            | App introduction/entry screen                              |
| Authentication | Login              | User sign-in                                               |
| Authentication | Register           | New account creation                                       |
| Authentication | Forgot Password    | Request a password reset code                              |
| Authentication | OTP Verify (Email) | Confirm email address via one-time code                    |
| Authentication | OTP Verify (Reset) | Confirm identity via one-time code before password reset   |
| Authentication | Reset Password     | Set a new password after OTP verification                  |
| Onboarding     | Add Device         | Device discovery entry point                               |
| Onboarding     | Provision Device   | Assign a name and register the device to the account       |
| Onboarding     | Wi-Fi Setup        | Enter Wi-Fi credentials for the device during provisioning |
| Core           | Home               | Overview dashboard summarizing all registered devices      |
| Core           | Devices            | List of all registered devices with live status            |
| Core           | Device Overview    | Live telemetry and relay control for a single device       |

|      |                   |                                                              |
|------|-------------------|--------------------------------------------------------------|
| Core | Device Automation | Timer scheduling and energy limit configuration for a device |
| Core | Device Settings   | Wi-Fi network update and device removal                      |
| Core | Fleet Analytics   | Aggregated energy analytics across all devices               |
| Core | Device Analytics  | Historical energy consumption chart for a single device      |
| Core | Alerts            | Notification inbox for energy-limit and connectivity events  |
| Core | Profile           | User account details and settings                            |

---

### 3.8 Testing and Evaluation Methodology

This section describes the methodology used to evaluate the completed prototype against Objective 5, covering measurement accuracy, remote control response time, and overall reliability under varying load conditions. The results of applying this methodology are reported in Chapter Four; this section specifies only the method, instrumentation, and metrics used.

#### 3.8.1 Measurement Accuracy

The measurement accuracy of the smart plug was evaluated by comparing the voltage reported by the PZEM-004T with measurements obtained from an external digital probe multimeter. The available reference instrument was capable of measuring AC voltage but could not provide independent current, power, or energy measurements without altering the load circuit. Consequently, independent accuracy validation was limited to the voltage measurement.

For each test load, ten corresponding reference and system voltage readings were recorded. The absolute error for each observation was calculated as:

$$AE_I = |V_{sys_i} - V_{ref_i}| \quad (3.13)$$

where  $V_{sys_i}$  represents the system voltage reported by the smart plug and  $V_{ref_i}$  represents the corresponding reference voltage measured by the probe multimeter.

The absolute percentage error for each observation was calculated as:

$$APE_I = \left| \frac{V_{sys_i} - V_{ref_i}}{V_{ref_i}} \right| \times 100 \quad (3.14)$$

The mean absolute error was calculated over the ten observations as:

$$MAE = \frac{1}{n} \sum_{i=1}^n |V_{sys_i} - V_{ref_i}| \quad (3.15)$$

where ( $n = 10$ ).

The mean percentage error was calculated as:

$$MAPE = \frac{1}{n} \sum_{i=1}^n \left| \frac{V_{sys_i} - V_{ref_i}}{V_{ref_i}} \right| \times 100 \quad (3.16)$$

The maximum absolute error was obtained as the largest absolute error among the ten observations, while the standard deviation of the absolute errors was calculated to characterize the variation in measurement deviation.

The voltage measurements were obtained under three representative loads: a soldering iron, a pressing iron, and a phone charger. The PZEM-004T current, active power, power factor, and energy measurements were recorded during these tests as system measurements but were not treated as independently validated accuracy measurements because no suitable reference instrumentation was available for these quantities.

### 3.8.2 Remote Control Response Time

Response time is measured as the elapsed interval between the timestamp at which a relay command is published from the mobile application and the timestamp at which the corresponding relay state change is confirmed, by physical observation and by the device's own state-change publication on the smartplug/{device\_id}/relay/state topic. This is repeated over multiple trials under normal network conditions to obtain an average and worst-case response time.

### 3.8.3 Reliability Under Varying Load Conditions

System reliability is assessed by operating the prototype continuously across a range of resistive and inductive household loads, verifying that relay switching, sensor readings, and the energy-limit cutoff logi behave consistently as load characteristics change. This includes deliberately testing behavior during a simulated network outage to confirm the local button-triggered control path remains functional independent of MQTT connectivity.

## CHAPTER FOUR

### RESULTS AND DISCUSSION

#### 4.1 Introduction

This chapter presents the results of implementing and testing the smart WiFi plug system described in Chapter 3. The results are organized around the hardware and software implementation outcomes, followed by the results of applying the three-part testing methodology defined in Section 3.8: energy monitoring accuracy, remote control response time, and reliability under varying load conditions. Each results section is followed by a brief discussion of what the results indicate; the chapter closes by evaluating the completed system against the five objectives stated in Section 1.4.

#### 4.2 Hardware Implementation

The hardware design described in Section 3.4 was assembled and tested on physical hardware. Plates 4.1 to 4.3 document the completed build.

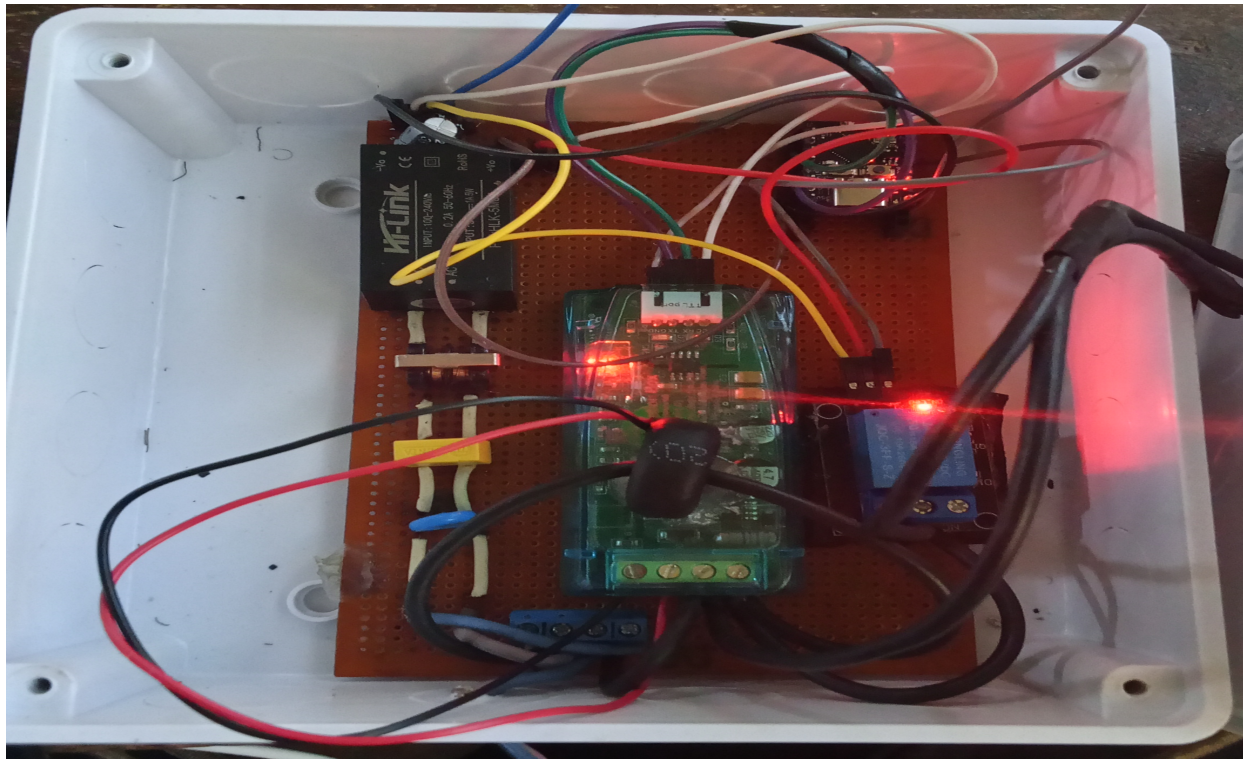


Plate 4.1: Populated perfboard before enclosure, showing component placement



Plate 4.2: Fully assembled unit inside the single-gang back box



Plate 4.3: Assembled unit connected and powered, ready for testing

### 4.3 Firmware and Software Implementation

The firmware described in Section 3.5 was flashed to the ESP32-C3 and verified to execute both operating modes correctly.

The FastAPI backend and PostgreSQL database were deployed on the Pxxl hosting platform, providing the cloud infrastructure for the application's API and persistent data storage. The deployed FastAPI service was verified through its interactive Swagger UI documentation, which was used to inspect and test the implemented API endpoints. The Swagger interface is accessible at <https://smart-wifi-socket.pxxlspace.cv/docs>. This confirmed that the deployed backend was operational and that the API endpoints could be accessed successfully. Figure 4.4 below shows a screenshot of the Swagger UI API documentation.

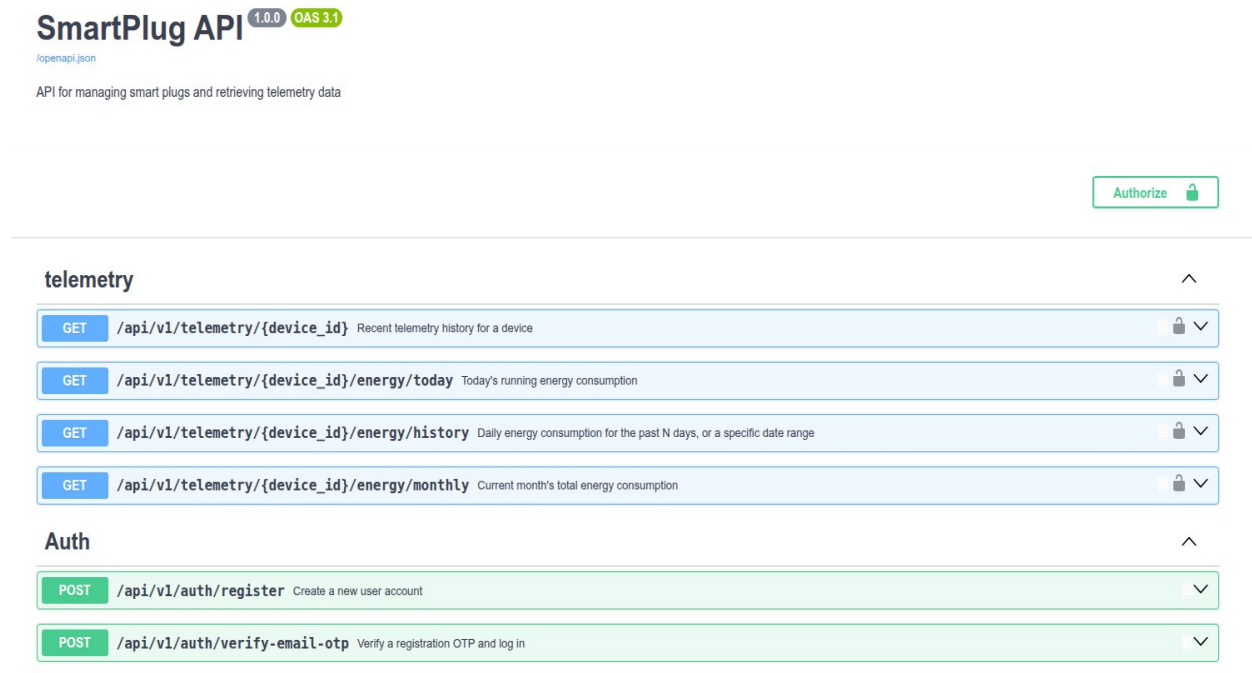


Figure 4.1: API Swagger UI Documentation Screenshot

The mobile application was verified against live hardware readings and event triggers, rather than static screen inventory described in Section 3.7.4. Figures 4.4 to 4.15 present key screens that demonstrate this live, populated functionality.

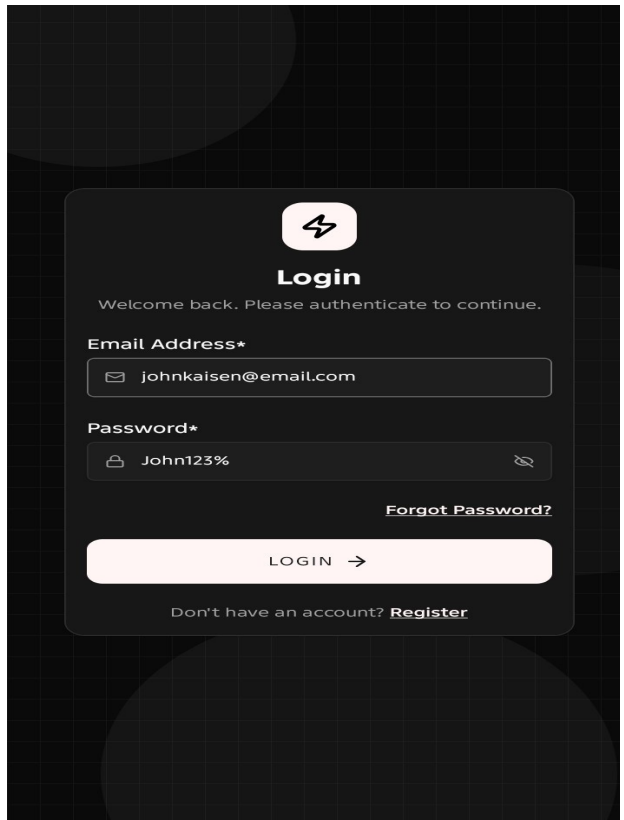


Figure 4.2: Login screen

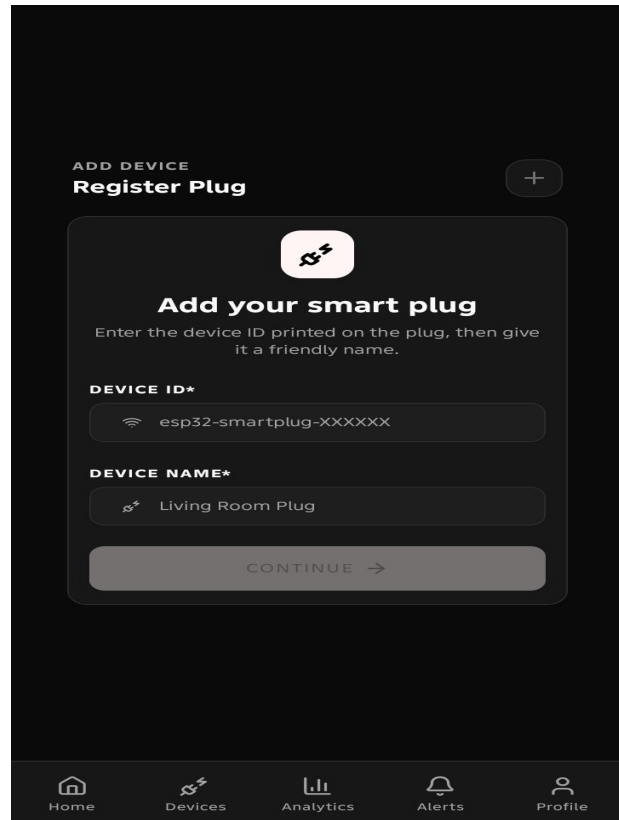


Figure 4.3: Add device screen

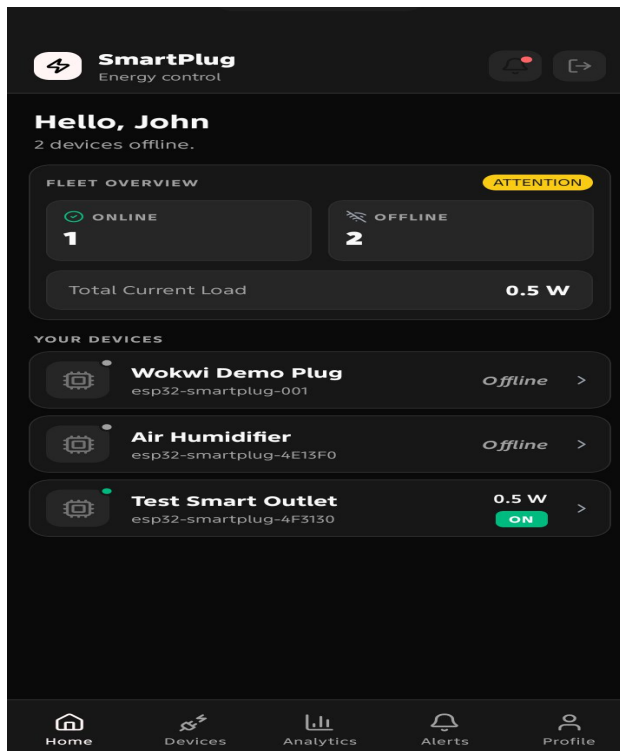


Figure 4.4: Home screen

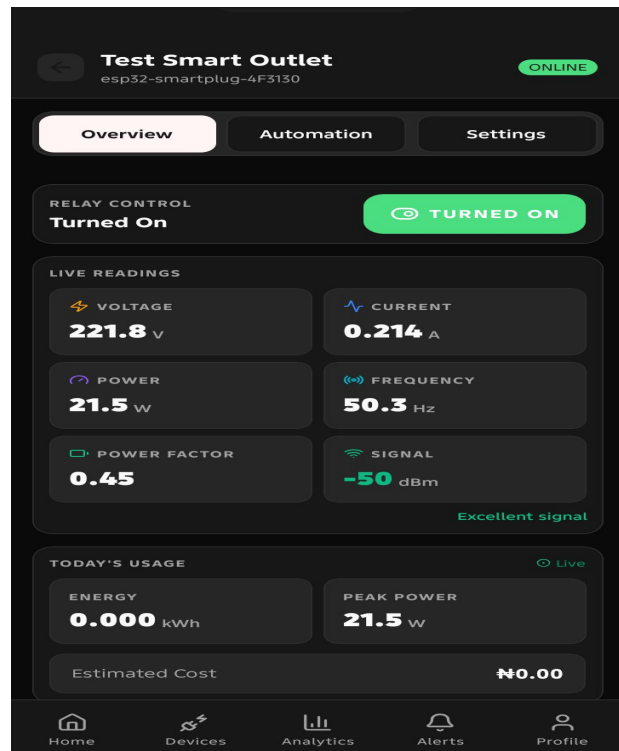


Figure 4.5: Device overview screen

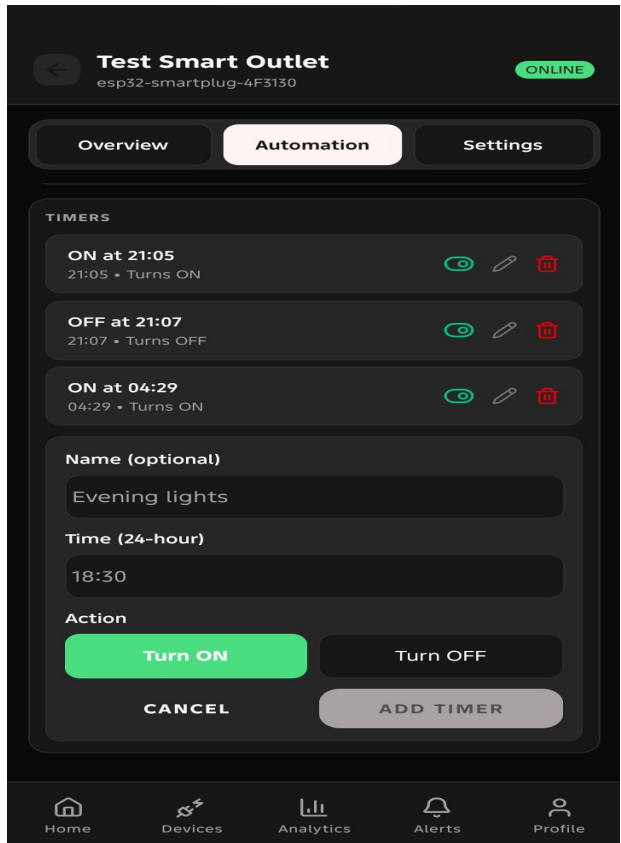


Figure 4.6: Device automation screen

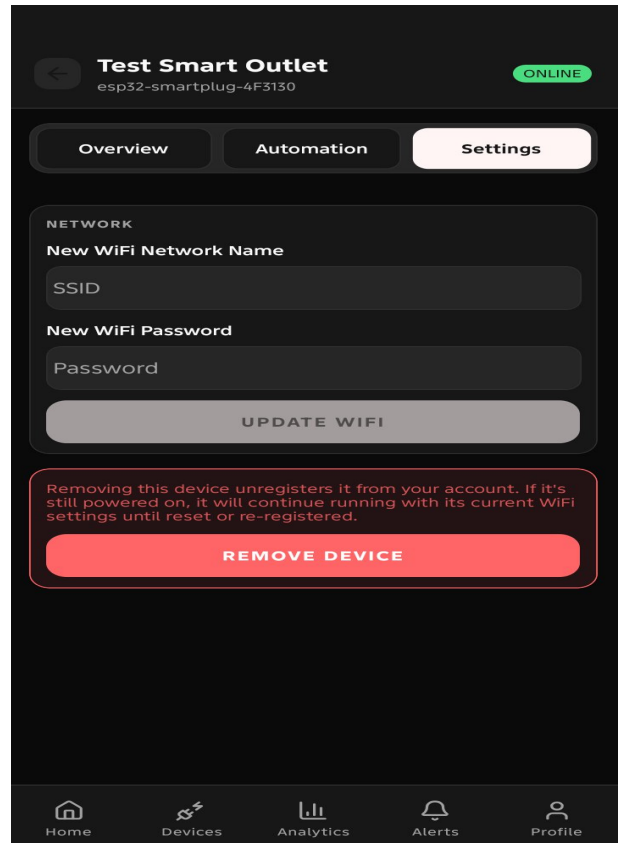


Figure 4.7: Device settings screen

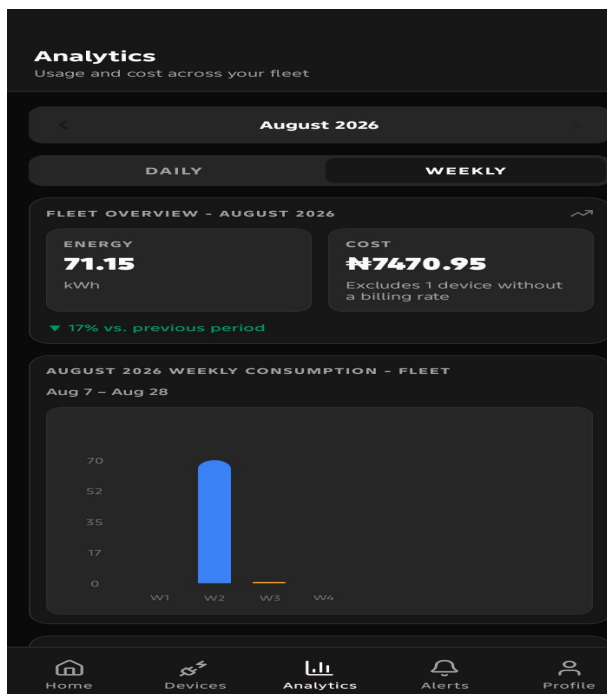


Figure 4.8: Fleet analytics screen

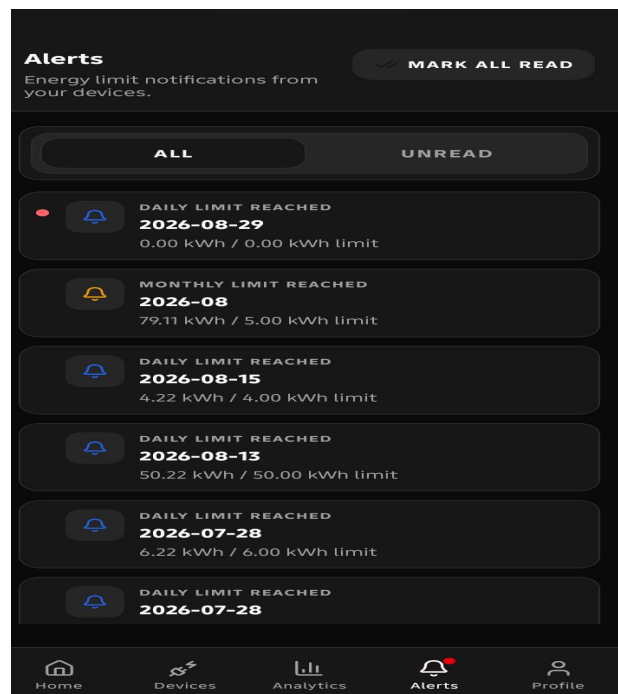


Figure 4.9: Alerts screen

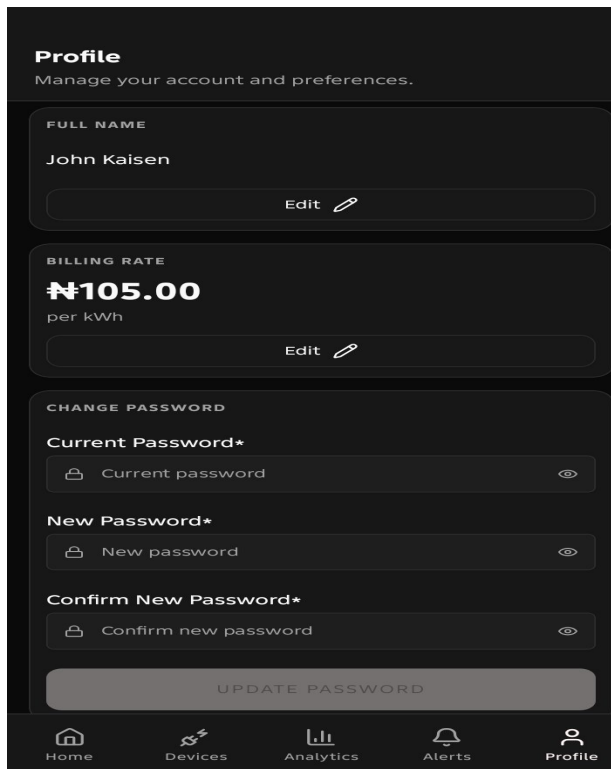


Figure 4.10: Profile screen

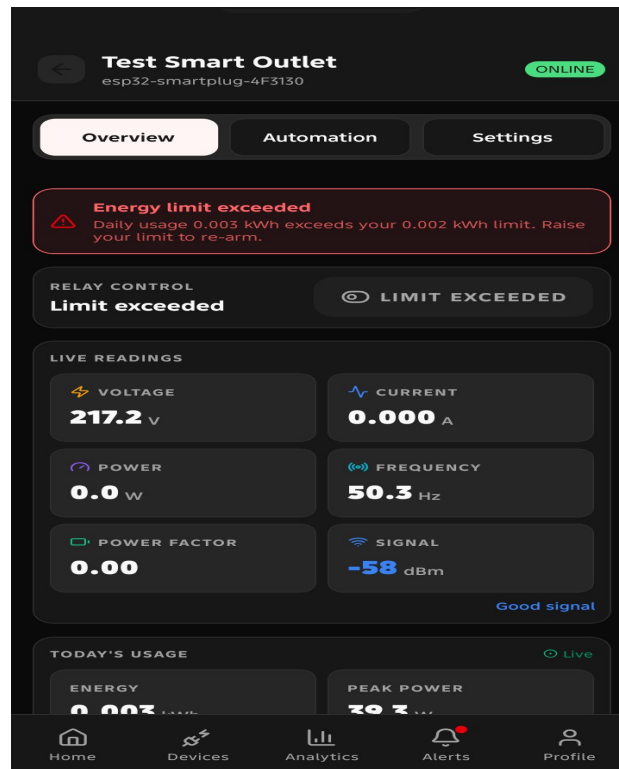


Figure 4.11: Energy limit triggered

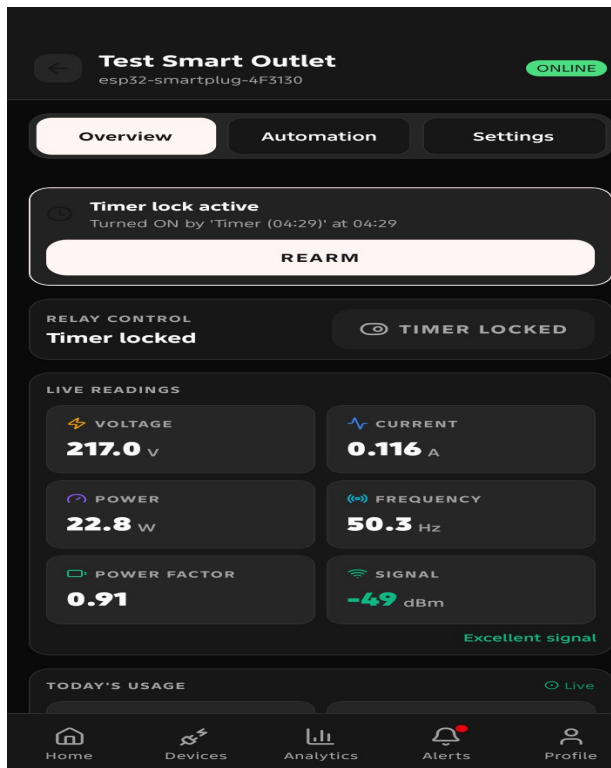


Figure 4.12: Timer triggered

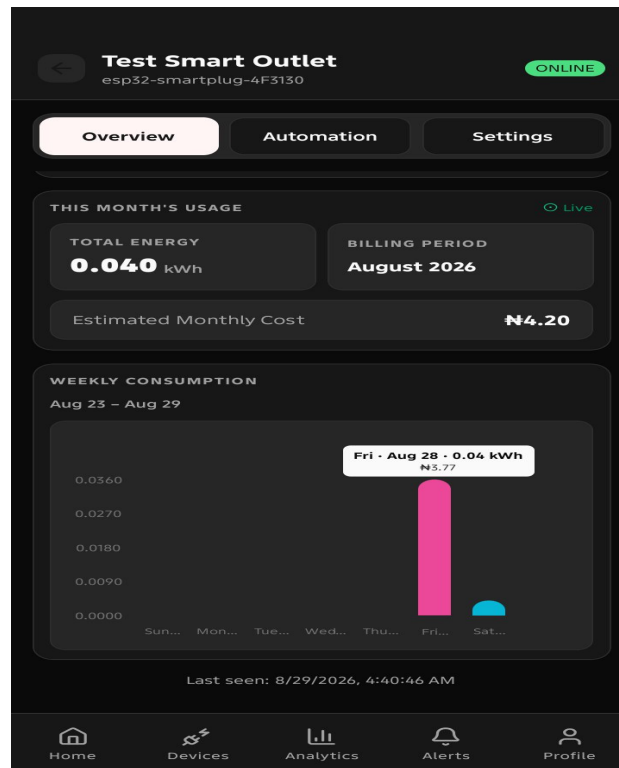


Figure 4.13: Device weekly usage screen

#### 4.4 Test Environment and Equipment

All tests reported in this chapter were conducted using the following reference equipment and conditions which are described in Table 4.1 below.

Table 4.1: Test equipments used and environment

| Item                             | Detail                                                                                                                                                                                    |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reference measurement instrument | Digital probe multimeter used to read AC voltage.<br><br>No clamp meter or plug-in wattmeter was available, which limits independent verification of current and power to the test loads. |
| Test loads used                  | Soldering iron (rated 60W, 220V, resistive); Pressing iron (rated 1200W, 220-240V, resistive); Phone charger (rated 12W, SMPS/electronic load, tested while actively charging).           |
| Wi-Fi network conditions         | Router Model: Infinix X6840 Smartphone Hotspot<br><br>Network: MTN 4G LTE<br><br>Location: FUTA Northgate<br><br>Upload Speed: 3.08 Mbps<br><br>Download Speed: 5.72 Mbps                 |

#### 4.5 Measurement and Performance Evaluation

Voltage was independently measured against a digital probe multimeter at each of ten sampled timestamps per load, allowing accuracy to be reported as a mean and spread across ten samples rather than a single point estimate. Current, power, power factor, and energy have no available independent reference instrument and are therefore reported as system measurements describing operating behaviour, not validated accuracy. This distinction is maintained strictly throughout this section: voltage is quantitatively validated against an external reference, while current, power, power factor, and energy are recorded observations of what the smart plug itself measured.

##### 4.5.1 Voltage Measurement Accuracy

The voltage measurement accuracy was evaluated using the procedure described in Section 3.8.1. Ten corresponding reference and system voltage measurements were obtained for each of the three

test loads. The calculated absolute and percentage errors are presented alongside the measured values in Tables 4.2 to 4.4.

Table 4.2: Soldering iron reference versus system voltage

| Sample | Time<br>(UTC+1) | Reference<br>Voltage (V) | System<br>Voltage (V) | Absolute<br>Error (V) | Absolute<br>Error (%) | Percentage |
|--------|-----------------|--------------------------|-----------------------|-----------------------|-----------------------|------------|
| 1      | 22:46:21        | 198                      | 194.7                 | 3.3                   | 1.67                  |            |
| 2      | 22:46:53        | 198                      | 196.2                 | 1.8                   | 0.91                  |            |
| 3      | 22:47:30        | 198                      | 195                   | 3                     | 1.52                  |            |
| 4      | 22:47:57        | 198                      | 195.1                 | 2.9                   | 1.47                  |            |
| 5      | 22:48:22        | 198                      | 194.9                 | 3.1                   | 1.57                  |            |
| 6      | 22:48:55        | 198                      | 195                   | 3                     | 1.52                  |            |
| 7      | 22:49:24        | 198                      | 194.4                 | 3.6                   | 1.82                  |            |
| 8      | 22:49:48        | 198                      | 195.2                 | 2.8                   | 1.41                  |            |
| 9      | 22:50:08        | 195                      | 191.8                 | 3.2                   | 1.64                  |            |
| 10     | 22:50:33        | 199                      | 195.1                 | 3.9                   | 1.96                  |            |

Table 4.3: Pressing iron reference versus system voltage

| Sample | Time<br>(UTC+1) | Reference<br>Voltage (V) | System<br>Voltage (V) | Absolute<br>Error (V) | Absolute<br>Error (%) | Percentage |
|--------|-----------------|--------------------------|-----------------------|-----------------------|-----------------------|------------|
| 1      | 22:28:05        | 192                      | 190.7                 | 1.3                   | 0.68                  |            |
| 2      | 22:31:15        | 180                      | 176.9                 | 3.1                   | 1.72                  |            |
| 3      | 22:34:28        | 192                      | 190                   | 2                     | 1.04                  |            |
| 4      | 22:35:04        | 180                      | 177.4                 | 2.6                   | 1.44                  |            |
| 5      | 22:35:55        | 180                      | 177.7                 | 2.3                   | 1.28                  |            |
| 6      | 22:36:26        | 178                      | 174                   | 4                     | 2.25                  |            |
| 7      | 22:37:23        | 194                      | 191.1                 | 2.9                   | 1.5                   |            |
| 8      | 22:38:03        | 182                      | 180.4                 | 1.6                   | 0.88                  |            |
| 9      | 22:41:45        | 184                      | 181.4                 | 2.6                   | 1.41                  |            |

|    |          |     |       |     |      |
|----|----------|-----|-------|-----|------|
| 10 | 22:43:10 | 197 | 194.5 | 2.5 | 1.27 |
|----|----------|-----|-------|-----|------|

Table 4.4: Phone charger reference versus system voltage

| Sample | Time<br>(UTC+1) | Reference<br>Voltage (V) | System<br>Voltage (V) | Absolute<br>Error (V) | Absolute<br>Error (%) | Percentage |
|--------|-----------------|--------------------------|-----------------------|-----------------------|-----------------------|------------|
| 1      | 22:55:22        | 201                      | 198.4                 | 2.6                   | 1.29                  |            |
| 2      | 22:55:50        | 201                      | 198.2                 | 2.8                   | 1.39                  |            |
| 3      | 22:56:18        | 200                      | 197.2                 | 2.8                   | 1.4                   |            |
| 4      | 22:56:55        | 200                      | 197.3                 | 2.7                   | 1.35                  |            |
| 5      | 22:57:28        | 203                      | 198.7                 | 4.3                   | 2.12                  |            |
| 6      | 22:57:57        | 202                      | 198.9                 | 3.1                   | 1.54                  |            |
| 7      | 22:58:27        | 203                      | 198.5                 | 4.5                   | 2.22                  |            |
| 8      | 22:58:50        | 202                      | 198.7                 | 3.3                   | 1.63                  |            |
| 9      | 22:59:13        | 201                      | 197.6                 | 3.4                   | 1.69                  |            |
| 10     | 22:59:40        | 201                      | 197.6                 | 3.4                   | 1.69                  |            |

The resulting error statistics are summarized in Table 4.5.

Table 4.5: Voltage error statistics across all three loads

| Load           | Mean Absolute<br>Error (V) | Max Absolute<br>Error (V) | Standard.<br>Deviation<br>(V) | Mean Absolute Percentage<br>Error (%) |
|----------------|----------------------------|---------------------------|-------------------------------|---------------------------------------|
| Soldering iron | 3.06                       | 3.90                      | 0.53                          | 1.55                                  |
| Pressing iron  | 2.49                       | 4.00                      | 0.73                          | 1.35                                  |
| Phone charger  | 3.29                       | 4.50                      | 0.62                          | 1.63                                  |

#### 4.5.2 Load Measurement

The smart plug's load-measurement performance was evaluated using the voltage, current, active power, power factor, and accumulated energy values reported by the PZEM-004T during operation

with the three test loads. Since suitable independent reference instruments were not available for current, power, power factor, and energy, the results in this section represent the electrical parameters measured by the developed system rather than independently validated accuracy measurements.

The system measurements obtained while operating the soldering iron is presented in Table 4.6.

Table 4.6: Soldering iron(rated 60W, 220V) system measurement

| Sample | Time (UTC+1) | Voltage (V) | Current (A) | Power (W) | Power Factor |
|--------|--------------|-------------|-------------|-----------|--------------|
| 1      | 22:46:21     | 194.7       | 0.121       | 23.4      | 1            |
| 2      | 22:46:53     | 196.2       | 0.106       | 20.7      | 1            |
| 3      | 22:47:30     | 195         | 0.1         | 19.3      | 1            |
| 4      | 22:47:57     | 195.1       | 0.096       | 18.6      | 1            |
| 5      | 22:48:22     | 194.9       | 0.094       | 18.3      | 0.99         |
| 6      | 22:48:55     | 195         | 0.093       | 18        | 1            |
| 7      | 22:49:24     | 194.4       | 0.092       | 17.7      | 0.99         |
| 8      | 22:49:48     | 195.2       | 0.092       | 17.9      | 1            |
| 9      | 22:50:08     | 191.8       | 0.09        | 17.2      | 0.99         |
| 10     | 22:50:33     | 195.1       | 0.091       | 17.8      | 1            |

The corresponding measurements for the pressing iron is presented in Table 4.7.

Table 4.7: Pressing iron(rated 1200W, 220 to 240V) system measurement

| Sample | Time(UTC+1) | Voltage (V) | Current (A) | Power (W) | Power Factor |
|--------|-------------|-------------|-------------|-----------|--------------|
| 1      | 22:28:05    | 190.7       | 3.991       | 718.7     | 1            |
| 2      | 22:31:15    | 176.9       | 3.946       | 698.1     | 1            |
| 3      | 22:34:28    | 190         | 0           | 0         | 0            |
| 4      | 22:35:04    | 177.4       | 3.972       | 704.8     | 1            |
| 5      | 22:35:55    | 177.7       | 3.969       | 705.1     | 1            |
| 6      | 22:36:26    | 174         | 3.89        | 677       | 1            |
| 7      | 22:37:23    | 191.1       | 0           | 0         | 0            |

|    |          |       |       |       |   |
|----|----------|-------|-------|-------|---|
| 8  | 22:38:03 | 180.4 | 4.023 | 725.7 | 1 |
| 9  | 22:41:45 | 181.4 | 4.046 | 733.7 | 1 |
| 10 | 22:43:10 | 194.5 | 0     | 0     | 0 |

The measurements obtained from the phone charger is presented in Table 4.8.

Table 4.8: Phone charger(rated 12W, 180 to 240V) system measurements

| Sample | Time (UTC+1) | Voltage (V) | Current (A) | Power (W) | Power Factor |
|--------|--------------|-------------|-------------|-----------|--------------|
| 1      | 22:55:22     | 198.4       | 0.087       | 11        | 0.64         |
| 2      | 22:55:50     | 198.2       | 0.085       | 10.8      | 0.64         |
| 3      | 22:56:18     | 197.2       | 0.085       | 10.7      | 0.64         |
| 4      | 22:56:55     | 197.3       | 0.084       | 10.7      | 0.65         |
| 5      | 22:57:28     | 198.7       | 0.083       | 10.8      | 0.65         |
| 6      | 22:57:57     | 198.9       | 0.083       | 10.7      | 0.65         |
| 7      | 22:58:27     | 198.5       | 0.084       | 10.5      | 0.64         |
| 8      | 22:58:50     | 198.7       | 0.084       | 10.6      | 0.64         |
| 9      | 22:59:13     | 197.6       | 0.083       | 10.4      | 0.63         |
| 10     | 22:59:40     | 197.6       | 0.083       | 10.5      | 0.63         |

#### 4.6 Remote Control Response Time

Following the methodology in Section 3.8.2, relay command response time was measured across ten repeated trials under the network conditions described in Table 4.9.

Table 4.9: Command response time

| Trial | Command Sent (UTC+1) | Relay Confirmed (UTC+1) | Response Time (ms) |
|-------|----------------------|-------------------------|--------------------|
| 1     | 04:09:05.000         | 4:09:05.826             | 826                |
| 2     | 04:09:37.000         | 4:09:38.004             | 1004               |
| 3     | 04:10:12.000         | 4:10:12.592             | 592                |
| 4     | 04:10:38.000         | 4:10:39.379             | 1379               |
| 5     | 04:11:08.000         | 4:11:08.952             | 952                |

|         |              |             |        |
|---------|--------------|-------------|--------|
| 6       | 04:11:33.000 | 4:11:34.604 | 1604   |
| 7       | 04:12:02.000 | 4:12:03.453 | 1453   |
| 8       | 04:12:36.000 | 4:12:36.838 | 838    |
| 9       | 04:12:58.000 | 4:12:59.520 | 1520   |
| 10      | 04:13:25.000 | 4:13:26.187 | 1187   |
| Average | -            | -           | 1135.5 |
| Minimum | -            | -           | 592    |
| Maximum | -            | -           | 1604   |

Response times ranged from 592ms to 1604ms, averaging 1135.5ms, over a mobile 4G hotspot connection rather than a fixed broadband line. This range is dominated by round-trip network latency between the mobile application, the cloud-hosted broker, and the device, rather than by processing delay on the ESP32-C3 itself.

#### 4.7 System Reliability Under Varying Load Conditions

Following the methodology in Section 3.8.3, the system was tested against the three loads used throughout this chapter to verify consistent relay switching, sensor stability, and cutoff logic. The reliability under the various loads is summarized in Table 4.10.

Table 4.10: System reliability under various loads

| Load Type                         | Rated Power (W) | Relay Switching | Sensor Reading Stable |
|-----------------------------------|-----------------|-----------------|-----------------------|
| Soldering iron (resistive)        | 60              | <i>Yes</i>      | <i>Yes</i>            |
| Pressing iron (resistive)         | 1200            | <i>Yes</i>      | <i>Yes</i>            |
| Phone charger (SMPS, 12 charging) | 12              | <i>Yes</i>      | <i>Yes</i>            |

#### 4.8 Discussion of Results Against Project Objectives

Table 4.11 summarizes how the results presented in this chapter address each objective stated in Section 1.4.

Table 4.11: Summary of project results against objectives.

| S/N | Objective                            |          | Result Summary                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----|--------------------------------------|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | ESP32/PZEM-004T architecture         | hardware | The ESP32-C3 and PZEM-004T based hardware was successfully assembled, powered, and connected as designed (Section 4.2), confirming the hardware architecture specified in Objective 1 was implemented and is functional on physical hardware.                                                                                                                                                                                                        |
| 2   | Wi-Fi/MQTT communication             |          | Wi-Fi and MQTT-based communication was verified end-to-end: the firmware established a TLS-secured MQTT connection and executed both operating modes correctly (Section 4.3), and relay commands were successfully transmitted and confirmed over MQTT across all ten response-time trials, averaging 1135.5ms under a mobile 4G connection (Section 4.6), confirming the communication architecture specified in Objective 2 functions as designed. |
| 3   | Relay switching isolation/protection | with     | Relay switching was tested across all three available loads (soldering iron, pressing iron, phone charger), with successful relay switching and stable sensor readings observed in every case (Section 4.7), confirming the optically isolated relay design (Section 3.4.5) performed reliably under real load conditions spanning a 1200W resistive load down to a 12W electronic load.                                                             |
| 4   | Mobile monitoring dashboard          |          | The mobile application's monitoring dashboard was verified against live hardware readings                                                                                                                                                                                                                                                                                                                                                            |

| S/N | Objective              | Result Summary                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|     |                        | and event triggers, including real-time telemetry display and remote relay control (Section 4.3), confirming the mobile monitoring and control functionality specified in Objective 4 is implemented and operational.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 5   | Testing and evaluation | Collectively, Sections 4.5-4.7 confirm the system was tested and evaluated as required by Objective 5: voltage measurement accuracy was validated against an independent reference across three loads with a consistent, small systematic bias (Section 4.5.1); current, power, and energy behaviour were characterized as system observations given the available instrumentation (Section 4.5.2-4.5.3); remote control response time was measured and found consistent with expected network-dominated latency (Section 4.6); and system reliability was assessed across resistive and electronic loads (Section 4.7). Wi-Fi connection stability, originally scoped within this objective, was not tested within the project timeframe (Section 4.9). |

#### 4.9 Limitations of Testing

First, only a digital probe multimeter (AC voltage only) was available for reference measurement; current, power, power factor, and energy (Section 4.5.2) are reported as system measurements without independent validation, and the energy accumulation cross-check (Section 4.5.3) is an internal consistency check, not an external accuracy validation.

Second, no genuinely inductive (AC motor-driven) load was available among the test equipment used (Section 4.7); all tested loads were either resistive or a switch-mode electronic load, so system behaviour under a load with power factor meaningfully below 1.0 from motor reactance specifically was not tested, though the phone charger's stable 0.63-0.65 power factor (Section 4.5.4) provides partial evidence of correct power factor sensing on a genuinely non-unity-PF load.

## CHAPTER FIVE

### CONCLUSION AND RECOMMENDATIONS

#### 5.1 Conclusion

This project set out to design and implement an IoT-based smart WiFi plug for remote energy monitoring and control, built around an ESP32-C3 microcontroller and a PZEM-004T energy monitoring module, communicating over MQTT to a cloud broker and backend, with control and monitoring provided through a mobile application. The five objectives stated in Section 1.3 have each been addressed and evaluated against real hardware, with results reported in Chapter 4.

The hardware architecture was successfully assembled and verified functional, with the ESP32-C3 and PZEM-004T correctly measuring voltage, current, power, frequency, and power factor, and communicating over the confirmed Modbus RTU protocol described in Section 3.4.4. Wi-Fi connectivity and MQTT-based communication were established and verified end-to-end, with TLS-secured broker communication and consistent relay command delivery across all tested trials. The relay-based load switching circuitry, protected by the optocoupler isolation and surge-protection components described in Section 3.4, performed reliably across all three tested loads, with no switching failures or sensor instability observed. The mobile application's monitoring dashboard was verified against live hardware data, successfully displaying real-time readings and enabling remote control during testing.

Testing and evaluation, the subject of Objective 5, produced a mixed but informative picture. Voltage measurement showed a consistent, small systematic bias of 1.3% to 1.6% across all three test loads, a result stable enough in direction and magnitude to suggest a fixed calibration or wiring offset rather than random sensor error. Current, power, and energy readings could not be independently validated due to the absence of a clamp meter or wattmeter, and are reported in this project as system observations rather than accuracy-confirmed results; where internal cross-checks were possible, results were consistent for two of three loads, with the third (the pressing iron) showing a discrepancy attributable to thermostatic load cycling interacting with sparse sampling intervals, not sensor malfunction. Remote control response time averaged 1135.5ms under mobile 4G network conditions, a figure dominated by network round-trip latency rather than device processing delay.

Taken together, these results indicate that the system meets its core functional objectives: it reliably measures and reports electrical parameters, switches load safely under real conditions, communicates securely and consistently over MQTT, and provides working remote monitoring and control through a mobile interface. The project's most significant limitation is not a failure of any implemented component, but a constraint of the testing process itself: without a clamp meter, wattmeter, or genuinely inductive test load, several aspects of Objective 5 (independent current/power/energy validation, and behavior under a reactive load) could only be partially addressed. Wi-Fi connection stability, originally scoped within Objective 5, was not tested within the timeframe available for this project. These limitations are stated explicitly in Section 4.9 rather than treated as resolved, and are carried forward into the recommendations below.

## **5.2 Recommendations for Future Work**

Based on the results and limitations identified in this project, the following are recommended for future development or subsequent evaluation of this system:

- a. Independent instrumentation for current, power, and energy validation. Future testing should use a clamp meter or plug-in wattmeter capable of independently measuring current and power, allowing the accuracy claims made for voltage in this report to be extended with equal confidence to the system's other measured parameters.
- b. Testing under a genuinely inductive load. No AC motor-driven load was available during this project's testing. Evaluating the system's power factor and current sensing under a real inductive load (a fan, pump, or compressor) would close a gap this project could only partially address using a switch-mode charger as an approximate substitute.
- c. Extension to multi-user and shared-device access, allowing a single smart plug to be monitored or controlled by more than one authorized user, which was outside this project's scope but a natural extension for shared or family living arrangements.
- d. Over-the-air (OTA) firmware updates, removing the current requirement for physical access to update device firmware, which would materially improve maintainability for any deployment beyond a single prototype unit.

## REFERENCES

- Aarti, M., Trupti, T., Nikita, P., and Khushi, K. (2026), Smart home automation system using Esp32, *International Journal of Science, Strategic Management and Technology*, 2, 5, 1–4, <https://doi.org/10.55041/ijst.v2i5.320> (Accessed August 18, 2026).
- Ahmed, D., Ali, S. Z., Khan, F. B., and Faruki, M. H. (2024), Design and implementation of an ESP32-based smart home automation system with environmental monitoring and automated controls, *IOSR Journal of Electronics and Communication Engineering*, 19, 6, 23–28, <https://doi.org/10.9790/2834-1906012328> (Accessed August 18, 2026).
- Alam, M. (2025), DIY AC energy meter using PZEM-004T & Arduino, *How To Electronics*, <https://how2electronics.com/diy-ac-energy-meter-using-pzem-004t-arduino/> (Accessed August 30, 2026).
- Al-Boghdady, A., Wassif, K., and El-Ramly, M. (2021), The presence, trends, and causes of security vulnerabilities in operating systems of IoT's low-end devices, *Sensors*, 21, 7, 2329, <https://doi.org/10.3390/s21072329> (Accessed August 18, 2026).
- Albraheem, L., Alajlan, H., Aljenedal, N., Alkhair, L. A., and Gwead, S. B. (2023), An IoT-based smart plug energy monitoring system, *International Journal of Advanced Computer Science and Applications*, 14, 10, 1038–1046, <https://doi.org/10.14569/ijacsa.2023.0141038> (Accessed August 18, 2026).
- Almani, D., and Furnell, S. (2025), Assessing the security vulnerabilities and countermeasures of connected and smart devices, *European Conference on Cyber Warfare and Security*, 24, 1, 3550, <https://doi.org/10.34190/eccws.24.1.3550> (Accessed August 18, 2026).
- Amirkhanov, B., Amirkhanova, G., Kunelbayev, M., Adilzhanova, S., and Tokhtassyn, M. (2025), Evaluating HTTP, MQTT over TCP and MQTT over WebSocket for digital twin applications: A comparative analysis on latency, stability, and integration, *International Journal of Innovative Research and Scientific Studies*, 8, 1, 679–694, <https://doi.org/10.53894/ijirss.v8i1.4414> (Accessed August 18, 2026).
- Anusha, K. (2024), Internet of Things (IoT) based home automation using ESP-32, *International Journal for Research in Applied Science and Engineering Technology*, 12, 5, 1672–1676, <https://doi.org/10.22214/ijraset.2024.61912> (Accessed August 18, 2026).
- Bajracharya, S., Paudel, S., Lama, P., and Upadhyay, R. (2025), Smart multiplug: IoT-based energy monitoring and remote-control system for sustainable power management, *International Journal on Engineering Technology*, 2, 2, 250–255, <https://doi.org/10.3126/injet.v2i2.78651> (Accessed August 18, 2026).

Bakhshi, T., Ghita, B., and Kuzminykh, I. (2024), A review of IoT firmware vulnerabilities and auditing techniques, *Sensors*, 24, 2, 708, <https://doi.org/10.3390/s24020708> (Accessed August 18, 2026).

Bhaganagare, S., Chavan, S., Gavali, S., and Godase, V. (2025a), The vocal cortex of the smart home: A review of ESP32-based control and automation architectures, *International Journal of Advanced Research in Science, Communication and Technology*, 5, 4, 1071–1079, <https://doi.org/10.48175/ijarsct-29460> (Accessed August 18, 2026).

Bhaganagare, S., Chavan, S., Gavali, S., and Godase, V. (2025b), Voice controlled home automation using ESP32 and IoT-based cloud integration, *International Journal of Advanced Research in Science, Communication and Technology*, 5, 4, 1411–1420, <https://doi.org/10.48175/ijarsct-29478> (Accessed August 18, 2026).

Bhosale, S., Nalogal, L., Golande, P., and Wable, A. (2022), Design smart home control system based on Android using IOT, *International Journal for Research in Applied Science and Engineering Technology*, 10, 12, <https://doi.org/10.22214/ijraset.2022.48393> (Accessed August 19, 2026).

Blanco-Novoa, Ó., Fernández-Caramés, T., Fraga-Lamas, P., and Castedo, L. (2017), An electricity price-aware open-source smart socket for the Internet of Energy, *Sensors*, 17, 3, 643, <https://doi.org/10.3390/s17030643> (Accessed August 18, 2026).

CircuitSchools Staff. (2026), Build an IoT AC energy meter with PZEM-004T & ESP32 web server, *CircuitSchools*, <https://www.circuitschools.com/build-an-iot-ac-energy-meter-with-pzem-004t-esp32-web-server/> (Accessed August 30, 2026).

Dhaou, I. (2023), Design and implementation of an Internet-of-Things-enabled smart meter and smart plug for home-energy-management system, *Electronics*, 12, 19, 4041, <https://doi.org/10.3390/electronics12194041> (Accessed August 18, 2026).

Dutta, P. K., El-Kenawy, S. M., Ali, G., and Dhoska, K. (2023), An energy consumption monitoring and control system in buildings using Internet of Things, *Babylonian Journal of Internet of Things*, 2023, 38–47, <https://doi.org/10.58496/bjiot/2023/006> (Accessed August 18, 2026).

Eldho, K. J., John, T. J., Jose, D. V., and Balakrishnan, R. V. (2025), Optimizing data transfer speed and the performance evaluation of MQTT in IoT: A study on MQTT for atmospheric condition monitoring, *Journal of Information Systems Engineering and Management*, 10, 39s, 231–244, <https://doi.org/10.52783/jisem.v10i39s.7144> (Accessed August 18, 2026).

Elorbany, K., Bayilmis, C., and Balta, S. (2021), A smart plug equipped with IoT technologies for energy management of electrical appliances, *Sakarya University Journal of Computer and Information Sciences*, 4, 3, 347–353, <https://doi.org/10.35377/saucis.04.03.863272> (Accessed August 18, 2026).

Enache, B. A., Banica, C. K., and Bogdan, A. (2023), Performance analysis of MQTT over WebSocket for IoT applications, *The Scientific Bulletin of Electrical Engineering Faculty*, 23, 1, 46–49, <https://doi.org/10.2478/sbeef-2023-0008> (Accessed August 18, 2026).

Gde, I., Sangka, N., Sunaya, N., Sajayasa, I. M., Widharma, G. S., et al. (2025), Design an IoT-based smart power outlet integrating digital technology for real-time electrical monitoring, *Contemporary Journal of Applied Sciences*, 3, 10, 649–660, <https://doi.org/10.55927/cjas.v3i10.91> (Accessed August 18, 2026).

Gomes, L., Sousa, F., and Vale, Z. (2018), An intelligent smart plug with shared knowledge capabilities, *Sensors*, 18, 11, 3961, <https://doi.org/10.3390/s18113961> (Accessed August 19, 2026).

Gomes, L., Sousa, F., Pinto, T., and Vale, Z. (2019), A residential house comparative case study using market available smart plugs and EnAPLugs with shared knowledge, *Energies*, 12, 9, 1647, <https://doi.org/10.3390/en12091647> (Accessed August 19, 2026).

Gomez, C., Chessa, S., Fleury, A., Roussos, G., and Preuveneers, D. (2019), Internet of Things for enabling smart environments: A technology-centric perspective, *Journal of Ambient Intelligence and Smart Environments*, 11, 1, 23–43, <https://doi.org/10.3233/ais-180509> (Accessed August 19, 2026).

Hasanah, M., Putri, D. I. H., and Pratama, H. P. (2025), IoT-based smart plug with real-time energy measurement optimization and adaptive current cutoff, *Jurnal Elektronika dan Telekomunikasi*, 25, 1, 46–54, <https://doi.org/10.55981/jet.673> (Accessed August 18, 2026).

Hassan, S. A. Z., and Eassa, A. M. (2022), A proposed architecture for smart home systems based on IoT, context-awareness and cloud computing, *International Journal of Advanced Computer Science and Applications*, 13, 6, 96–105, <https://doi.org/10.14569/ijacsa.2022.0130612> (Accessed August 19, 2026).

Hercog, D., Lerher, T., Truntič, M., and Težak, O. (2023), Design and implementation of ESP32-based IoT devices, *Sensors*, 23, 15, 6739, <https://doi.org/10.3390/s23156739> (Accessed August 18, 2026).

Hintaw, A. J. (2025), Performance analysis of MQTT protocol in IoT environments: Impact of payload size and QoS on key metrics, *Technium: Romanian Journal of Applied Sciences and Technology*, 28, 43–57, <https://doi.org/10.47577/technium.v28i.12649> (Accessed August 18, 2026).

Jara Ochoa, H. J., Peña, R., Mezquita, Y. L., Gonzalez, E., and Camacho-León, S. (2023), Comparative analysis of power consumption between MQTT and HTTP protocols in an IoT platform designed and implemented for remote real-time monitoring of long-term cold chain transport operations, *Sensors*, 23, 10, 4896, <https://doi.org/10.3390/s23104896> (Accessed August 18, 2026).

Jiang, X., Dutta, P., Culler, D., and Stoica, I. (2007), Micro power meter for energy monitoring of wireless sensor networks at scale, *2007 6th International Symposium on Information Processing in Sensor Networks*, 186–195, <https://doi.org/10.1145/1236360.1236386> (Accessed August 18, 2026).

K, K. K. (2024), Smart power monitoring system using IoT, *International Journal of Scientific Research in Engineering and Management*, 8, 5, <https://doi.org/10.55041/ijrem33763> (Accessed August 18, 2026).

Kabir, M. A. A., Elmedany, W., and Sharif, M. S. (2023), Securing IoT devices against emerging security threats: Challenges and mitigation techniques, *Journal of Cyber Security Technology*, 7, 4, 199–223, <https://doi.org/10.1080/23742917.2023.2228053> (Accessed August 18, 2026).

Kashyap, N., and Kashyap, S. K. (2025), Real time energy management in Wireless Sensor Networks (WSNs), *Journal of Advances and Scholarly Researches in Allied Education*, 22, 1, 1–6, <https://doi.org/10.29070/gk9cs262> (Accessed August 18, 2026).

Khadar, S., Benkhider, A., Djelbane, A., and Draoui, A. (2024), Smart plugs for home automation systems, *Studies in Engineering and Exact Sciences*, 5, 2, 1–16, <https://doi.org/10.54021/seesv5n2-788> (Accessed August 18, 2026).

Kim, H.-H., and Yoo, J. (2022), Analysis of security vulnerabilities for IoT devices, *Journal of Information Processing Systems*, 18, 4, 489–499, <https://doi.org/10.3745/jips.03.0178> (Accessed August 18, 2026).

Kolanur, C. B., Banakar, R., and Rajneesh, G. (2021), Design of IoT based platform development for smart home appliances control, *Journal of Physics: Conference Series*, 1969, 1, 012052, <https://doi.org/10.1088/1742-6596/1969/1/012052> (Accessed August 18, 2026).

Malik, G., and P., P. (2023), IoT security: The vulnerabilities in internet of things devices and securing IoT ecosystem, *International Journal of Science and Research Archive*, 8, 1, 93–99, <https://doi.org/10.30574/ijjsra.2023.8.1.0093> (Accessed August 18, 2026).

Mhetre, J. G., Kumbharkar, R. V., and Pawar, S. R. (2026), Smart energy meter, *International Journal of Science, Strategic Management and Technology*, 2, 4, 22–27, <https://doi.org/10.55041/ijjsmt.v2i4.022> (Accessed August 19, 2026).

Mishra, B., Mishra, B., and Kertész, A. (2021), Stress-testing MQTT brokers: A comparative analysis of performance measurements, *Energies*, 14, 18, 5817, <https://doi.org/10.3390/en14185817> (Accessed August 18, 2026).

Mishra, M., and Guru, S. (2026), Performance evaluation of MQTT, CoAP and HTTP protocols for application-specific IoT scenarios, *International Journal for Research in Applied Science and Engineering Technology*, 14, 2, 883–888, <https://doi.org/10.22214/ijraset.2026.77541> (Accessed August 18, 2026).

Murat, K., Topyła, D., Zdulski, K., Marzecki, M., Bieniasz, J., Paczesny, D., and Szczypiorski, K. (2024), Security analysis of low-budget IoT smart home appliances embedded software and connectivity, *Electronics*, 13, 12, 2371, <https://doi.org/10.3390/electronics13122371> (Accessed August 18, 2026).

Nebrida, A. P., Amador, C. D., Madiam, C. M., and Ranche, G. J. S. (2023), Development of smart meter to monitor real time energy consumption for sustainability, *International Journal of Sustainable Construction Engineering and Technology*, 14, 4, 1–10, <https://doi.org/10.30880/ijscet.2023.14.04.003> (Accessed August 19, 2026).

Ningbo Peacefair Electronic Technology Co., Ltd. (2019), PZEM-004T V3.0 user manual [Datasheet], *InnovatorsGuru*, <https://innovatorsguru.com/wp-content/uploads/2019/06/PZEM-004T-V3.0-Datasheet-User-Manual.pdf> (Accessed August 30, 2026).

Oh, J. (2020), IoT-based smart plug for residential energy conservation: An empirical study based on 15 months' monitoring, *Energies*, 13, 15, 4035, <https://doi.org/10.3390/en13154035> (Accessed August 18, 2026).

Oo, Z., Lai, T. W., and Moe, A. (2022), IoT based home automation system using a REST API architecture, *European Journal of Technic*, 12, 2, 131–138, <https://doi.org/10.36222/ejt.1018131> (Accessed August 19, 2026).

Rajasekar, V., and Rajkumar, S. (2023), A study on Internet of Things devices vulnerabilities using Shodan, *International Journal of Computing*, 22, 2, 149–158, <https://doi.org/10.47839/ijc.22.2.3084> (Accessed August 18, 2026).

Rao, C. K., Sahoo, S. K., and Yanine, F. F. (2025), Development of intelligent smart energy monitoring systems for renewable energy source using Proteus-8 environment and validated with experimentation, *Journal of Electrical Systems and Information Technology*, 12, 1, 12, <https://doi.org/10.1186/s43067-025-00202-3> (Accessed August 19, 2026).

Risfendra, R., Ananda, G. F., and Stephanus, A. (2021), Internet of Things on electrical energy monitoring using multi-electrical parameter sensors, *Motivection: Journal of Mechanical, Electrical and Industrial Engineering*, 3, 1, 1–10, <https://doi.org/10.46574/motivection.v3i1.79> (Accessed August 19, 2026).

Ruminot, N., Estevez, C., and Montejo Sánchez, S. (2023), A novel approach of a low-cost voltage fault injection method for resource-constrained IoT devices: Design and analysis, *Sensors*, 23, 16, 7180, <https://doi.org/10.3390/s23167180> (Accessed August 18, 2026).

S, P., and K, A. S. (2023), Smart remote controlling system of home appliances, *International Journal for Multidisciplinary Research*, 5, 4, 4227–4234, <https://doi.org/10.36948/ijfmr.2023.v05i04.4227> (Accessed August 18, 2026).

Sánchez-Sutil, F., and Cano-Ortega, A. (2022), Smart plug for monitoring and controlling electrical devices with a wireless communication system integrated in a LoRaWAN, *Expert Systems with Applications*, 213, Part A, 118976, <https://doi.org/10.1016/j.eswa.2022.118976> (Accessed August 30, 2026).

Santos, A., Duggan, G., Davis, J., and Zimmerle, D. (2023), A cautionary note on using smart plugs for research data acquisition, *e-Prime - Advances in Electrical Engineering, Electronics and Energy*, 4, 100137, <https://doi.org/10.1016/j.prime.2023.100137> (Accessed August 19, 2026).

Sayanna, M., Bharath, M., Naveen, B., Kumar, T. S., and Kumar, P. S. (2026), Remote-controlled home appliance system via MQTT and ESP8266, *International Journal of AI Electronics and Nexus Energy*, 2, 1, 50–57, <https://doi.org/10.64751/vr4rm250> (Accessed August 18, 2026).

Sharma, P. (2023), Zigbee based Wireless Sensor Network for smart energy meter, *International Journal of Recent Technology and Engineering (IJRTE)*, 12, 3, 23–26, <https://doi.org/10.35940/ijrte.c7861.0912323> (Accessed August 18, 2026).

Shvaika, D. I., Shvaika, A. I., and Artemchuk, V. (2025), MQTT broker architectural enhancements for high-performance P2P messaging: TBMQ scalability and reliability in

distributed IoT systems, *IoT*, 6, 3, 34, <https://doi.org/10.3390/iot6030034> (Accessed August 18, 2026).

Sifawa, H. A., Tambawal, A. B., and S/Tudu, S. M. (2026), Performance evaluation of IoT protocols in resource-constraints smart home networks, *International Journal of Science for Global Sustainability*, 12, 1, 48–60, <https://doi.org/10.57233/ijsgs.v12i1.1012> (Accessed August 18, 2026).

Silva, D. M. A., Carvalho, L., Soares, J. A. M., and Sofia, R. C. (2021), A performance analysis of Internet of Things networking protocols: Evaluating MQTT, CoAP, OPC UA, *Applied Sciences*, 11, 11, 4879, <https://doi.org/10.3390/app11114879> (Accessed August 18, 2026).

Sk, N. A. (2022), A smart web based energy monitoring system for residential electricity consumption, *International Journal of Scientific Research in Engineering and Management*, 6, 5, <https://doi.org/10.55041/ijssrem15959> (Accessed August 19, 2026).

Suryadevara, N., and Biswal, G. (2019), Smart plugs: Paradigms and applications in the smart city and smart grid, *Energies*, 12, 10, 1957, <https://doi.org/10.3390/en12101957> (Accessed August 19, 2026).

Unisa, S. A. (2022), Smart home control, *International Journal for Research in Applied Science and Engineering Technology*, 10, 5, 4545–4551, <https://doi.org/10.22214/ijraset.2022.45451> (Accessed August 19, 2026).

Villegas, M. D., Paredes, E. D., Arévalo, J., Quito Carrión, A., Pillajo, R., Sánchez, A. C., and Proaño, P. (2025), Smart automation for residential spaces with PLC-ESP32 architecture, *The XXXIII Conference on Electrical and Electronic Engineering*, 115, 1, 7, <https://doi.org/10.3390/engproc2025115007> (Accessed August 18, 2026).